

PRISM: Distribution-free Adaptive Computation of Matrix Functions for Accelerating Neural Network Training

Shenghao Yang^{1,2} Zhichao Wang^{1,2} Oleg Balabanov^{1,2} N. Benjamin Erichson^{1,3} Michael W. Mahoney^{1,2,3}

Abstract

Matrix functions such as square root, inverse roots, and orthogonalization play a central role in preconditioned gradient methods for neural network training. This has motivated the development of iterative algorithms that avoid explicit eigendecompositions and rely primarily on matrix multiplications, making them well suited for modern GPU accelerators. We present PRISM¹ (Polynomial-fitting and Randomized Iterative Sketching for Matrix functions computation), a general framework for accelerating iterative algorithms for computing matrix functions. PRISM combines adaptive polynomial approximation with randomized sketching: at each iteration, it fits a polynomial surrogate to the current spectrum via a sketched least-squares problem, adapting to the instance at hand with minimal overhead. We apply PRISM to accelerate Newton–Schulz-like iterations for matrix square roots and orthogonalization, which are core primitives in machine learning. Unlike prior methods, PRISM requires no explicit spectral bounds or singular value estimates; and it adapts automatically to the evolving spectrum. Empirically, PRISM accelerates training when integrated into Shampoo and Muon optimizers.

1. Introduction

Matrix functions are extensively used in scientific and engineering applications, and they are of increasing interest in machine learning (ML). Applications range from computational fluid dynamics (Ndjunga, 2008; Castro et al., 2016)

and computational chemistry (Lin et al., 2009b;a) to optimal transport (Janati et al., 2020; Minh, 2022), Gaussian processes and Bayesian inference (Mallasto & Feragen, 2017; Pleiss et al., 2020), uncertainty quantification (Chen et al., 2019), computer vision (Wang et al., 2021; Song et al., 2021; 2023), and fast optimizers for training deep neural networks (Carlson et al., 2015; Gupta et al., 2018; Yao et al., 2020; 2021; Jordan et al., 2024; Ahn et al., 2025). As such, the ability to compute simple matrix functions, such as square root, inverse root, and orthogonalization (polar decomposition), in a fast and numerically stable manner, can lead to substantial improvements.

Among the many alternatives for computing matrix functions, iterative algorithms that rely primarily on General Matrix Multiplications (GEMMs) are particularly attractive for GPU-accelerated computing environments (Volkov & Demmel, 2008; Markidis et al., 2018; Yan et al., 2020; Amsel et al., 2026; Grishina et al., 2025). Compared with algorithms based on the singular value decomposition (SVD), or those that involve some form of matrix inversion, GEMMs have much better scaling with respect to the size of a matrix. Because of this, iterative algorithms of the form $\mathbf{X}_{k+1} = F_k(\mathbf{X}_k)$, where computing $F_k(\mathbf{X}_k)$ is fast on accelerators, have recently received increasing interest (Amsel et al., 2026; Grishina et al., 2025; Kim et al., 2025). A simple example is to restrict F_k to be a polynomial $p_k^{(d)}$ of fixed degree d at k -th iteration, in which case evaluating $F_k(\mathbf{X}_k) = p_k^{(d)}(\mathbf{X}_k)$ only requires GEMMs.

This general approach has received interest in ML due to the effectiveness of the Muon optimizer (Jordan et al., 2024) for training neural networks. Subsequent work (Amsel et al., 2026; Grishina et al., 2025) explored the acceleration of the initial convergence of Newton–Schulz-like iterative methods for the polar factor $\mathbf{U}\mathbf{V}^T$ of a matrix $\mathbf{A}$, where $\mathbf{U}$ and $\mathbf{V}$ consist of the left and right singular vectors of $\mathbf{A}$, respectively. In particular, for $d \in \{3, 5\}$ and any $K \geq 1$, they showed how to construct a polynomial p^* by composing degree- d polynomials such that:

$$p^* = \arg \min_{p = p_K^{(d)} \circ p_{K-1}^{(d)} \circ \dots \circ p_1^{(d)}} \max_{\substack{\mathbf{A} \in \mathbb{R}^{m \times n}; \\ \sigma(\mathbf{A}) \subseteq [\ell, u]}} \|p(\mathbf{A}) - \mathbf{U}\mathbf{V}^T\|_2,$$

where $\sigma(\mathbf{A})$ denotes the set of singular values of $\mathbf{A}$. Thus,

¹International Computer Science Institute, Berkeley CA, USA
²University of California, Berkeley CA, USA ³Lawrence Berkeley National Laboratory, Berkeley CA, USA. Correspondence to: Shenghao Yang <shenghao.yang@berkeley.edu>, Zhichao Wang <zhichao.wang@berkeley.edu>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

¹Code is available at <https://github.com/s-h-yang/PRISM>

if the largest and smallest singular values of $\mathbf{A}$ are known a priori (which, in general, is not the case), the methods proposed by Amsel et al. (2026) and Grishina et al. (2025) provide optimal convergence with respect to the spectral norm error $\|\mathbf{X}_k - \mathbf{U}\mathbf{V}^T\|_2$. In addition to polar decomposition, Kim et al. (2025) considers computing matrix roots and inverses via a Monte-Carlo Tree Search (MTCS) method to construct iterative algorithms of the form $\mathbf{X}_{k+1} = r_k(\mathbf{X}_k)$, where r_k is either a polynomial (i.e., Newton-Schulz-like) or a rational function (i.e., Newton-like); and, when the underlying distribution of singular values is known, they demonstrated strong performance.

While promising, these recently-introduced methods suffer from several disadvantages, which currently limit the broader applicability of this approach. First, they tend to be solved for a single problem, rather than for a broader class of problems, as is more common in numerical analysis (Higham et al., 2005). In ML, they are just applied and evaluated for one specific use case, e.g., within the Muon optimizer where one only needs to compute the polar factor of the gradient matrix. This can lead to strong performance in one setting, but it obscures the broader applicability of the methodology, and it prevents a “cut-and-paste” approach of using this methodology to new problem classes and application domains. Second, these recently-introduced methods tend to be parameterized in terms of parameters that are themselves as difficult to compute as solving the original problem. In particular, in practice, we typically do not have prior knowledge of the distribution of singular values or even a tight interval that contains them. Obtaining good estimates on the largest and, in particular, the smallest singular values of a matrix can be as costly as computing its polar factor by using the original Newton-Schulz method.

To deal with this, Amsel et al. (2026) suggested fixing the range of singular values to $[\ell, u] = [10^{-3}, 1]$, when the computations are carried out in half precision. However, if the actual range of singular values is much narrower or wider than a predefined interval, which can easily happen, then the convergence behavior can degrade dramatically. An example of this is shown in Figure 1, where we compare Newton-Schulz variants for computing polar factor (i.e., orthogonalization) and square root. Observe that PolarExpress (Amsel et al., 2026) can even *slow down* the convergence of the classical Newton-Schulz, if there is a mismatch between the tightest interval that contains the initial singular values and the interval for which the method is optimized.

These shortcomings highlight the need for a principled adaptive approach for a broader class of problems that effectively generates polynomials $p_k^{(d)}(\cdot; \mathbf{X}_0)$ whose coefficients do not assume properties of the spectrum of its input matrix, but instead dynamically adjust to the spectrum. We introduce PRISM (Polynomial-fitting and Randomized Iterative

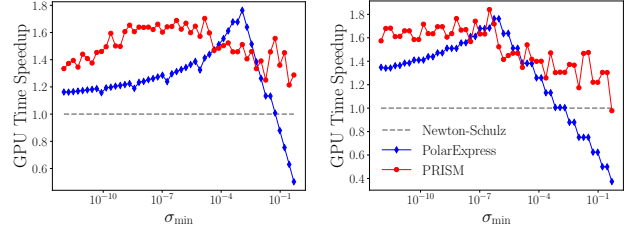


Figure 1. Speedup in GPU time over the classical Newton-Schulz for polar decomposition (left) and square root (right). We keep $\sigma_{\max} = 1$ and vary $\sigma_{\min} \in [10^{-12}, 1/2]$. The PolarExpress variant we use is optimized for $\sigma_{\min} = 10^{-3}$ for polar decomposition (and hence it is optimized for $\sigma_{\min} = 10^{-6}$ for square root). All algorithms are run until convergence. In both cases, we see a performance degradation as $\sigma_{\min}$ deviates from the one PolarExpress is optimized for. PRISM (this work) does not require or assume $\sigma_{\min}$ and has a stable speedup across the entire range.

Sketching for Matrix functions computation), a general framework for accelerating the computation of matrix functions via spectrum-adaptive polynomial updates (Chen et al., 2011) and randomized sketching (Murray et al., 2023).

PRISM targets GPU-friendly iterations and is designed to be broadly applicable, computationally efficient, and robust to variations in spectral structure.

Contributions. Summarizing our contributions:

- **General acceleration framework.** PRISM provides a unified, systematic approach for accelerating iterative algorithms for matrix functions, including Newton-Schulz methods for square roots, inverse roots, sign, and polar decomposition; Chebyshev method for the inverse; and inverse Newton for the inverse p -th root. See Table 1 for some iterative algorithms accelerated by PRISM.
- **Spectrum-adaptive without prior spectral knowledge.** PRISM dynamically fits polynomial updates to the evolving spectrum of the current iterate without requiring explicit eigenvalue or singular-value information. This leads to an instance-specific and *distribution-free* acceleration of classical iterative algorithms.
- **Efficient randomized polynomial fitting with guarantees.** Using randomized sketching, PRISM reduces the overhead cost of polynomial fitting to $O(n^2 \log n)$, which is nearly negligible compared to the $O(n^3)$ cost of matrix multiplications. At the same time, it preserves the convergence behavior of the underlying iterations, both theoretically and empirically.
- **Empirical validation in neural network optimizers.** We show that PRISM effectively accelerates Newton-Schulz-like algorithms for square roots and polar decomposition on various input matrices, including those with a Marchenko-Pastur law (e.g., neural network weight matrices at initialization) or a heavy-tailed distribution (e.g.,

pre-trained models (Martin & Mahoney, 2021; Hodgkinson et al., 2025)). When integrated into the Shampoo and Muon optimizers, PRISM efficiently accelerates training large neural networks in both cases.

2. Related Work

Iterative algorithms for matrix function computation. A large body of numerical linear algebra treats matrix functions via iterations that are dominated by matrix–matrix multiplies and thus well-suited to modern accelerators. Classical methods include Newton/Schulz and Padé-type schemes for matrix sign and polar decomposition (Kenney & Laub, 1991; Higham et al., 2004; 2005), stable iterations for the matrix square root and its variants (Higham, 1997), and later improvements using Zolotarev and Halley/QDWH-style rational approximants with careful stability analyses (Nakatsukasa & Freund, 2016; Nakatsukasa & Higham, 2012). These methods motivate designing polynomial or rational updates that (i) avoid explicit inverses or SVD, (ii) converge quickly over a prescribed spectral interval, and (iii) map cleanly to GEMM-dominant kernels (Fan et al., 2019; 2020; Arisaka & Li, 2023; Ndjinga, 2008; Kim et al., 2025).

Randomized/sketching methods for matrix functions and traces. Randomized numerical linear algebra (RandNLA) provides subspace embeddings and sketching primitives that reduce dimension, while approximately preserving the spectral structure (Mahoney, 2011; Woodruff, 2014; Drineas & Mahoney, 2018; Nelson & Nguyen, 2013). For quantities involving $f(A)$ or $\text{tr } f(A)$, stochastic trace estimation and Krylov/Lanczos quadrature are now standard: Hutchinson/Avron–Toledo and sharper sample-complexity bounds (Avron & Toledo, 2011; Roosta-Khorasani & Ascher, 2015; Cortinovis & Kressner, 2022), and SLQ for $\text{tr } f(A)$ with strong empirical and theoretical support (Ubaru et al., 2017; Yao et al., 2020). Variance-reduced estimators, such as Hutch++, further improve sample complexity in the PSD case and beyond (Meyer et al., 2021). Recently, Huang et al. (2025); Refael et al. (2025) applied randomized SVD to efficiently train neural networks.

Newton–Schulz iteration in deep learning. In large-scale training, GEMM-only iterations have reappeared as inner loops inside optimizers. Shampoo constructs Kronecker-factored preconditioners using (inverse) matrix square roots (Gupta et al., 2018); and AdaHessian uses randomized curvature (Hutchinson) to approximate Hessian–diagonals (Yao et al., 2021). More recently, Muon applied Newton–Schulz-based orthogonalization to momentum matrices to produce direction-only updates for matrix-shaped parameters (Jordan et al., 2024). Two follow-ups design iterations expressly for ML constraints: *PolarExpress* formulates a minimax-optimal polynomial update for the polar/sign problem with GPU-friendly stability and demonstra-

ble Muon gains (Amsel et al., 2026); and *CANS* (Chebyshev–Accelerated Newton–Schulz) uses Chebyshev polynomials and Remez optimization to accelerate early iterations and offer controlled approximate orthogonalization (Grishina et al., 2025). At scale, *DION* shows how to efficiently distribute orthonormalized updates in data-parallel/FSDP systems (Ahn et al., 2025). This line of work motivates adaptive, spectrum-aware polynomial updates that retain the hardware efficiency prized in ML training. However, all of these prior methods only focus on polar decomposition.

Other ML applications of iterative matrix functions. Beyond optimization, matrix functions (especially $A^{1/2}$ and $A^{-1/2}$) recur in areas such as computer vision and probabilistic ML. In global covariance pooling and second-order layers, Newton–Schulz/Taylor or Padé-based approximations are competitive with or superior to SVD in speed and accuracy (Li et al., 2018; Song et al., 2021; 2023). In Gaussian–process and Bayesian–optimization pipelines, fast actions of $K^{\pm 1/2}$ via iterative quadrature provide scalable alternatives to dense factorizations (Pleiss et al., 2020). Whitening/Coloring transforms for universal style transfer and de-correlated batch norm similarly rely on differentiable square roots/inverse square roots (Li et al., 2017; Huang et al., 2018; Song et al., 2023). Finally, Riemannian optimization on the Stiefel manifold uses polar-based retractions whose inner loops are identical to the iterations studied here (Absil et al., 2008; Grishina et al., 2025).

3. The PRISM Meta-algorithm

We now lay out the high-level structure of PRISM in the form of a meta-algorithm. The PRISM meta-algorithm starts by framing the design of existing or new algorithms as that of iterative polynomial approximation (Part I: Basic setup). Then, a principled acceleration scheme naturally arises, in which randomized sketching is used to efficiently improve polynomial approximation, leading to accelerated convergence behavior (Part II: Acceleration). A variety of classical iterative algorithms for matrix functions fit into the basic setup of PRISM. Consequently, all of these algorithms can be accelerated by deploying the acceleration techniques outlined in Part II of PRISM. In the next paragraph, we demonstrate how Newton–Schulz iterations are derived by following Part I of PRISM. Analogous derivations for Newton (Appendix A.2), Inverse Newton (Appendix A.3), and Chebyshev (Appendix A.4) methods are deferred to the appendix.

Deriving Newton–Schulz with PRISM Part I. Let $x \neq 0$ be such that $\text{sign}(x) = \text{sign}(a)$. Then, $\text{sign}(a) = \text{sign}(x) = x(x^2)^{-1/2} = x(1 - \xi)^{-1/2} = xf(\xi)$, where $f(\xi) = (1 - \xi)^{-1/2}$, and where $\xi = 1 - x^2$ measures how close x is to $\text{sign}(x) = \text{sign}(a)$. Therefore, the problem of approximating $\text{sign}(a)$ leads to that of approximating $f(\xi)$.

PRISM Meta-algorithm for Computing $T(\mathbf{A})$
Part I: Basic setup

1. Let x be an estimate of $T(a)$ where $a \in \mathbb{R}$. Write $T(a) = xf(\xi)$ for a residual function $\xi = \xi(x, a)$.
2. Set up the scalar iteration $x_{k+1} = x_k \cdot f_d(\xi(x_k, a))$, where $f_d(\xi)$ is the d -th order Taylor's expansion of $f(\xi)$ around $\xi = 0$.
3. To compute $T(\mathbf{A})$, run the matrix version,

$$\mathbf{X}_{k+1} = \mathbf{X}_k f_d(\mathbf{R}_k),$$
 where $\mathbf{R}_k = \xi(\mathbf{X}_k, \mathbf{A})$ is the residual matrix.

Part II: Acceleration

4. **Polynomial fitting:** To accelerate convergence, replace f_d with $g_d(\xi; \alpha) = f_{d-1}(\xi) + \alpha \xi^d$, iterate

$$\mathbf{X}_{k+1} = \mathbf{X}_k g_d(\mathbf{R}_k; \alpha_k^*),$$
 where $\alpha_k^* = \arg \min_{\alpha} \|\xi(\mathbf{X}_{k+1}, \mathbf{A})\|_F^2$ minimizes the residual norm.
5. **Sketching:** To maintain low cost at every iteration, use $\tilde{\alpha}_k = \arg \min_{\alpha} \|\mathbf{S} \xi(\mathbf{X}_{k+1}, \mathbf{A})\|_F^2$ in place of α_k^* , where $\mathbf{S}$ is a low-dimensional sketch matrix.

Using the d -th order Taylor polynomial $f_d(\xi)$ around $\xi = 0$, we obtain an iterative procedure $x_{k+1} = x_k f_d(\xi(x_k))$. The matrix version, $\mathbf{X}_{k+1} = \mathbf{X}_k f_d(\mathbf{R}_k)$, where $\mathbf{R}_k = \mathbf{I} - \mathbf{X}_k^2$, gives the generalized Newton-Schulz iteration for matrix sign. When $d = 1$, this reduces to the standard one, $\mathbf{X}_{k+1} = \frac{1}{2}\mathbf{X}_k - \frac{3}{2}\mathbf{X}_k^3$, which converges quadratically when initialized properly, e.g., when $\mathbf{X}_0 = \mathbf{A}/\|\mathbf{A}\|_F$. When $d \geq 2$, this leads to high-order variants with higher-order convergence rates. We note that this procedure was first described by [Kenney & Laub \(1991\)](#) to derive the more general Padé family of iterative rational methods for computing the matrix sign. In addition, due to the close relationship between iterative algorithms for computing the matrix sign, square roots, and polar factor ([Higham et al., 2004](#); [Higham, 1997](#)), the Newton-Schulz variants for computing square roots and polar decomposition can be derived analogously.

The role of Part I in PRISM is to provide a common ground so that different algorithms for computing different matrix functions can all be accelerated in a similar way as outlined in Part II, which serves as PRISM's main algorithmic component. Once a new or existing iterative algorithm is fitted into Part I of PRISM, such as the Newton-Schulz iterations we discussed above, as well as additional examples in Appendix A, one may apply Part II of PRISM to accelerate convergence at low cost.

Since Taylor's polynomial may not provide the best approx-

imation of the target function at individual eigenvalues of the residual matrix $\mathbf{R}_k$, a poor fit can consequently result in a slow initial convergence of the corresponding algorithm. PRISM Part II directly addresses this in the following way:

- In order to improve convergence (across iterations) of the algorithm, Step 4 of PRISM replaces the Taylor polynomial with one that better fits to the spectrum. We will require that the residual matrix $\mathbf{R}_k = \xi(\mathbf{X}_k, \mathbf{A})$ be symmetric, and hence by minimizing the squared Frobenius norm, α_k^* effectively fits the candidate polynomial $g_d(\xi; \alpha)$ on the eigenvalues of $\mathbf{R}_k$ by minimizing a (nonlinear) least-squares loss. We defer an in-depth discussion of this to Section 4, where we focus on the particular example of matrix sign computation. We will also explain why the candidate polynomial $g_d(\xi; \alpha)$ was chosen to take that particular form.
- In order to speed up the run time (of each iteration) of the algorithm, in Step 5 of PRISM, we use randomized sketching methods from RandNLA to significantly reduce the cost of least-squares polynomial fitting. This is essential to ensure the practicality of PRISM: it accelerates convergence by automatically adapting to the spectrum—without requiring any knowledge on the spectral distribution of the input matrix—at comparably negligible additional cost. We will show that appropriately chosen sketch matrices do not compromise the performance.

4. Matrix Sign: a Case Study on How PRISM Accelerates Convergence at Low Cost

In this section, we describe how the PRISM meta-algorithm can be applied to develop an accelerated Newton-Schulz iteration for computing matrix sign. PRISM applies more broadly, but we start here with the matrix sign function in order to present the core ideas in a single setting, without deviating to small algorithmic or technical differences. The matrix sign function is particularly interesting because iterative algorithms for matrix square roots and polar decomposition—two primitive matrix functions that arise in neural network optimizers—are closely related to sign computation, and analogous results readily hold for those algorithms (see Section 5). We will discuss how Part II of PRISM accelerates classical Newton-Schulz iterations. Although the derivations and illustrations presented in this section are specific to matrix sign, analogous results, such as why classical Newton-Schulz is slow and why PRISM accelerates them, hold more generally for other Newton-Schulz-like algorithms, including all those present in Table 1.

For a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, the matrix sign function is defined as $\text{sign}(\mathbf{A}) = \mathbf{A}(\mathbf{A}^2)^{-1/2}$. For our analysis we require that $\mathbf{A}^2$ is symmetric, which practically covers all relevant use cases and hence we will assume it true through-

out. To simplify notation, we also assume that $\|A\|_2 \leq 1$, with the understanding that such condition is easily satisfied by normalization $A \mapsto A/\|A\|_F$. As derived in Section 3, the Newton-Schulz iteration for matrix sign is

$$X_0 = A, R_k = I - X_k^2, X_{k+1} = X_k f_d(R_k), \quad (1)$$

and $f_d(\xi)$ is the d -th order Taylor approximation of $f(\xi) = (1 - \xi)^{-1/2}$ around $\xi = 0$. While $f_d(\xi)$ provides an accurate estimate of $f(\xi)$ near $\xi = 0$, the error increases rapidly as ξ approaches 1. In the early phase of Newton-Schulz, if X_k has eigenvalues close to 0, then the matrix polynomial $f_d(I - X_k^2)$ is not a good approximant of $f(I - X_k^2)$, and thus the convergence of (1) can be slow. To see this more clearly, consider $d = 1$ and the scalar sequence

$$x_{k+1} = x_k f_1(1 - x_k^2) = x_k(1 + \frac{1}{2}(1 - x_k^2)).$$

It is easy to verify that, for x_k close to 0,

$$1 - x_{k+1}^2 = \frac{3}{4}(1 - x_k^2)^2 + \frac{1}{4}(1 - x_k^2)^3 \approx 1 - \frac{9}{4}x_k^2,$$

where we used Taylor approximation around $x_k = 0$. This means that even though the sequence is still quadratically convergent since $|1 - x_{k+1}^2| \leq |1 - x_k^2|^2$, the initial convergence rate behaves much like a linear one with a relatively small constant around 9/4. An illustration is provided in Figure 2. The same observation generalizes to high-order Taylor series for $d \geq 1$ and matrix iterations.

4.1. Fitting Polynomials to Matrix Spectrum

In order to accelerate the convergence of (1), we replace the Taylor polynomial f_d with a different polynomial g_d by iteratively fitting it to the target function $f(\xi) = (1 - \xi)^{-1/2}$ over the spectrum of the current iterate. This is Step 4 of the PRIME meta-algorithm. Since the error $|f(\xi) - f_d(\xi)|$ is proportional to ξ^{d+1} , instead of fitting an entirely new polynomial, which can be difficult and costly, we consider the class of degree- d polynomials of the form

$$g_d(\xi; \alpha) = f_{d-1}(\xi) + \alpha \xi^d.$$

That is, we keep all but the coefficient of ξ^d the same and change α so that $g_d(\xi; \alpha)$ is a better fit to the data $\{\lambda_i, f(\lambda_i)\}_{i=1}^n$ in the least-squares sense, where λ_i denotes the i -th eigenvalue of the residual matrix $R = I - X^2$. We will discuss in more detail shortly, but let us start with a simple example on how this can accelerate convergence. Consider again the case with $d = 1$ and the scalar sequence $x_{k+1} = x_k f_1(1 - x_k^2)$. We have seen that if x_k is near 0 then $1 - x_{k+1}^2 \approx 1 - 2.25x_k^2$. If we replace $f_1(\xi) = 1 + \frac{1}{2}\xi$ with $g_1(\xi; 1) = 1 + \xi$, then for x_k close to 0,

$$1 - x_{k+1}^2 = (1 - x_k^2)^2 + (1 - x_k^2)^3 - (1 - x_k^2) \approx 1 - 4x_k^2.$$

This shows that with $\alpha_k = 1$, for x_k close to 0, we still maintain quadratic convergence $|1 - x_{k+1}^2| \leq |1 - x_k^2|^2$,

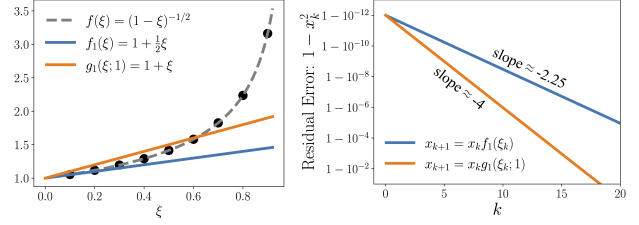


Figure 2. Better polynomial approximation leads to faster convergence. Left: Approximating $f(\xi)$ using its Taylor approximation $f_1(\xi)$ around $\xi = 0$ versus the alternative $g_1(\xi; 1)$. Right: The initial convergence behavior in residual error $\xi_k = 1 - x_k^2$ using the standard and “accelerated” Newton-Schulz, respectively, for $x_0 = 10^{-6}$. Using $g_1(\xi; 1)$ leads to an exponential speedup.

and although the local convergence behavior is still much like a linear one, the error $1 - x_{k+1}^2$ diminishes at a rate that is nearly *twice* as rapid. An illustration is provided in Figure 2, where we see that a better fit of $f(\xi)$ for $\xi \gg 0$ leads to a much faster convergence of the resulting sequence.

Step 4 of PRISM changes (1) into

$$X_0 = A, R_k = I - X_k^2, X_{k+1} = X_k g_d(R_k; \alpha_k^*), \quad (2)$$

$$\alpha_k^* = \arg \min_{\alpha \in [\ell, u]} \|I - X_k^2 g_d(R_k; \alpha)\|_F^2$$

$$= \arg \min_{\alpha \in [\ell, u]} \sum_{i=1}^n (1 - (1 - \lambda_{k,i}) g_d(\lambda_{k,i}; \alpha))^2, \quad (3)$$

where $\lambda_{k,1}, \lambda_{k,2}, \dots, \lambda_{k,n}$ are the eigenvalues of R_k . The last equality follows because A^2 is symmetric and $X_0 = A$, so R_k is symmetric for all k . Therefore, (3) fits the polynomial $g_d(x; \alpha)$ to the set of points $\{(\lambda_{k,i}, f(\lambda_{k,i}))\}_{i=1}^n$, and recall that $f(\xi) = (1 - \xi)^{-1/2}$. In (3), we impose an interval constraint $\alpha \in [\ell, u]$ to regularize the problem so that α_k^* will not be negatively affected by potential outliers. Without such a constraint, the resulting residual matrix still has a strictly smaller Frobenius norm, and hence the sequence generated by (2) still converges to $\text{sign}(A)$. However, it is important to ensure that our choice of α_k indeed accelerates the overall rate of convergence rather than making it slower. A natural condition we would like to guarantee is the sequence of residual matrices R_k should have a strictly decreasing spectral norm. Since the Taylor polynomial $f_d(\xi)$ approximates $f(\xi)$ from below, i.e. $f(\xi) - f_d(\xi) > 0$ for all $\xi \in (0, 1)$, if unconstrained, α^* can be unnecessarily large, causing an oscillating behavior in the spectral norm of the residual matrix, e.g. having $\|R_{k+1}\|_2 > \|R_k\|_2$, which can hurt the overall convergence rate. The interval $[\ell, u]$ we add to (3) should ensure that the resulting iteration in (2) converges at least as fast as the original one in (1). It turns out that there is a principled way to choose $[\ell, u]$ based on the resulting polynomial properties. For example, one may set $[\ell, u] = [1/2, 1]$ for $d = 1$ and $[\ell, u] = [3/8, 29/20]$ for $d = 2$, which cover the 3rd- and 5th-order Newton-Schulz

Table 1. PRISM-accelerated algorithms for computing a few primitive matrix functions that arise in neural network optimizers

Method	Target	Initialization	Iteration*	Residual
Newton-Schulz (3rd-order)	$\mathbf{A}^{1/2}$ $\mathbf{A}^{-1/2}$	$\mathbf{X}_0 = \mathbf{A}$ $\mathbf{Y}_0 = \mathbf{I}$	$\mathbf{X}_{k+1} = \mathbf{X}_k(\mathbf{I} + \alpha_k \mathbf{R}_k)$ $\mathbf{Y}_{k+1} = (\mathbf{I} + \alpha_k \mathbf{R}_k) \mathbf{Y}_k$	$\mathbf{R}_k = \mathbf{I} - \mathbf{X}_k \mathbf{Y}_k$
Newton-Schulz (5th-order)	$\mathbf{A}^{1/2}$ $\mathbf{A}^{-1/2}$	$\mathbf{X}_0 = \mathbf{A}$ $\mathbf{Y}_0 = \mathbf{I}$	$\mathbf{X}_{k+1} = \mathbf{X}_k(\mathbf{I} + \frac{1}{2} \mathbf{R}_k + \alpha_k \mathbf{R}_k^2)$ $\mathbf{Y}_{k+1} = (\mathbf{I} + \frac{1}{2} \mathbf{R}_k + \alpha_k \mathbf{R}_k^2) \mathbf{Y}_k$	$\mathbf{R}_k = \mathbf{I} - \mathbf{X}_k \mathbf{Y}_k$
Newton-Schulz (3rd-order)	$\mathbf{U} \mathbf{V}^T$	$\mathbf{X}_0 = \mathbf{A}$	$\mathbf{X}_{k+1} = \mathbf{X}_k(\mathbf{I} + \alpha_k \mathbf{R}_k)$	$\mathbf{R}_k = \mathbf{I} - \mathbf{X}_k^T \mathbf{X}_k$
Newton-Schulz (5th-ordNeer)	$\mathbf{U} \mathbf{V}^T$	$\mathbf{X}_0 = \mathbf{A}$	$\mathbf{X}_{k+1} = \mathbf{X}_k(\mathbf{I} + \frac{1}{2} \mathbf{R}_k + \alpha_k \mathbf{R}_k^2)$	$\mathbf{R}_k = \mathbf{I} - \mathbf{X}_k^T \mathbf{X}_k$
Coupled Inverse Newton	$\mathbf{A}^{-1/p}$ ($p \geq 1$)	$\mathbf{X}_0 = \mathbf{I}$ $\mathbf{M}_0 = \mathbf{A}$	$\mathbf{X}_{k+1} = \mathbf{X}_k(\mathbf{I} + \alpha_k \mathbf{R}_k)$ $\mathbf{M}_{k+1} = (\mathbf{I} + \alpha_k \mathbf{R}_k)^p \mathbf{M}_k$	$\mathbf{R}_k = \mathbf{I} - \mathbf{X}_k^p \mathbf{A}$
DB Newton	$\mathbf{A}^{1/2}$ $\mathbf{A}^{-1/2}$	$\mathbf{X}_0 = \mathbf{A}$ $\mathbf{Y}_0 = \mathbf{I}$	$\mathbf{X}_{k+1} = (1 - \alpha_k) \mathbf{X}_k + \alpha_k \mathbf{Y}_k^{-1}$ $\mathbf{Y}_{k+1} = (1 - \alpha_k) \mathbf{Y}_k + \alpha_k \mathbf{X}_k^{-1}$	$\mathbf{R}_k = \mathbf{I} - \mathbf{X}_k \mathbf{Y}_k$
Chebyshev	$\mathbf{A}^{-1}$	$\mathbf{X}_0 = \mathbf{A}^T$	$\mathbf{X}_{k+1} = \mathbf{X}_k(\mathbf{I} + \mathbf{R}_k + \alpha_k \mathbf{R}_k^2)$	$\mathbf{R}_k = \mathbf{I} - \mathbf{A} \mathbf{X}_k$

*The value of α_k depends on both the input data and the underlying algorithm, see PRISM meta-algorithm for how it is defined/computed.

variants, respectively. We formally state a convergence result in Theorem 4.1 for the case $d = 1$, and leave an analogous result to Appendix D where we discuss how $[\ell, u]$ should be chosen in detail.

Theorem 4.1. *Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be such that $0 < \|\mathbf{A}\|_2 \leq 1$ and $\mathbf{A}^2$ is symmetric. Let $\mathbf{X}_0 = \mathbf{A}$ and consider the sequence of matrices $\mathbf{X}_1, \mathbf{X}_2, \dots$ generated by (2) where α_k^* is determined by (3), with $d = 1$, $\ell = 1/2$ and $u = 1$. We have that $\mathbf{X}_k \rightarrow \text{sign}(\mathbf{A})$ and $\|\mathbf{I} - \mathbf{X}_k^2\|_2 \leq \|\mathbf{I} - \mathbf{A}^2\|_2^{2^{k-2}}$.*

Remark. The proof of Theorem 4.1, which we leave to Appendix C.2, is based on demonstrating that the polynomial $h(\xi, \alpha_k^*) = 1 - (1 - \xi)g_d(\xi; \alpha_k^*)^2$, which defines the recurrence relation in the residual space, maintains a good quadratic convergence behavior for all k and all possible initial eigenvalues $\{\lambda_{0,i}\}_{i=1}^n \subseteq [0, 1]^n$. The assumption that $\mathbf{A}^2$ is symmetric covers the case $\mathbf{A} = \begin{bmatrix} 0 & \mathbf{A}' \\ \mathbf{I} & 0 \end{bmatrix}$ where $\mathbf{A}'$ is symmetric. This will be useful later to apply Theorem 4.1 and get an analogous result for computing matrix square root. Theorem 4.1 indicates that for $d = 1$ and $[\ell, u] = [1/2, 1]$ in the computation of α_k^* , (1) converges at least as fast as the classical Newton-Schulz (Higham et al., 2005). The bounds $[\ell, u]$ depend solely on some favorable polynomial properties of $g_d(\xi; \alpha)$ and are independent of the spectrum of the input matrix. We refer the reader to Lemma C.1 for details. For the case $d \geq 2$, the same line of arguments can be applied to find reasonable choices for $[\ell, u]$. These bounds are universal in the sense that they are completely independent from the spectral properties (or any other characteristics such as the size) of the input matrix.

4.2. Fast Approximate Polynomial Fitting via Randomized Sketching

For the matrix sign iteration (as well as square roots and polar decomposition which we discuss later) in (2), the loss

function in (3) is a degree-4 polynomial in α_k , i.e.,

$$\begin{aligned} m(\alpha) &:= \|\mathbf{I} - \mathbf{X}_k^2 g_d(\mathbf{I} - \mathbf{X}_k^2; \alpha)^2\|_F^2 \\ &= c_0 + c_1 \alpha + c_2 \alpha^2 + c_3 \alpha^3 + c_4 \alpha^4. \end{aligned}$$

The coefficients of $m(\alpha)$ depend on d . For example, for $d = 1$ and $g_1(\xi; \alpha) = 1 + \alpha \xi$ we have $c_1 = \text{tr}(4\mathbf{R}_k^3 - 4\mathbf{R}_k^2)$, $c_2 = \text{tr}(6\mathbf{R}_k^4 - 10\mathbf{R}_k^3 + 4\mathbf{R}_k^2)$, $c_3 = \text{tr}(4\mathbf{R}_k^5 - 8\mathbf{R}_k^4 + 4\mathbf{R}_k^3)$, $c_4 = \text{tr}(\mathbf{R}_k^6 - 2\mathbf{R}_k^5 + \mathbf{R}_k^4)$. For general $d \geq 1$, computing these coefficients requires access to the diagonal entries of $\mathbf{R}_k^i$ for i up to $4d + 2$. We will discuss how to speed up this computation using randomized sketching in the following paragraphs, but once we know c_1, c_2, c_3, c_4 , minimizing $m(\alpha)$ can be done analytically by solving the cubic equation $m'(\alpha) = 0$. We provide more details in Appendix A.1.

In order to obtain the coefficients c_i 's of $m(\alpha)$, naively computing $\mathbf{R}_k^{4d+2}$ and then evaluating its trace requires at least $\log_2(4d + 2) \geq 2 + \log_2(d)$ matrix multiplications. This can be more expensive than executing a full iteration of classical Newton-Schulz, and thus rendering our acceleration scheme too costly. Ideally, we would like to obtain a good polynomial $g_d(\xi; \alpha_k)$ in sub-cubic time with respect to n . To accomplish this, we use randomized sketching and approximately minimize $m(\alpha)$ with a controlled error rate. A matrix $\mathbf{S} \in \mathbb{R}^{p \times n}$ is an (p, ϵ, δ) -Oblivious Subspace Embedding (OSE)² if for any fixed p -dimensional subspace $\mathbb{V} \subseteq \mathbb{R}^n$, with probability at least $1 - \delta$, for all $\mathbf{x} \in \mathbb{V}$ we have $(1 - \epsilon)\|\mathbf{x}\|_2^2 \leq \|\mathbf{S}\mathbf{x}\|_2^2 \leq (1 + \epsilon)\|\mathbf{x}\|_2^2$. Instead of (3), we choose α_k by solving the following problem:

$$\tilde{\alpha}_k = \arg \min_{\alpha \in [\ell, u]} \|\mathbf{S}_k (\mathbf{I} - \mathbf{X}_k^2 g_d(\mathbf{R}_k; \alpha)^2)\|_F^2, \quad (4)$$

where $\mathbf{S}_k \in \mathbb{R}^{p \times n}$ is an OSE. The loss function in (4) is

²Subspace embeddings were first introduced by Drineas et al. (2006); they were first used in data-oblivious form by Sarlós (2006); Drineas et al. (2011); and they were popularized in RandNLA by Woodruff (2014).

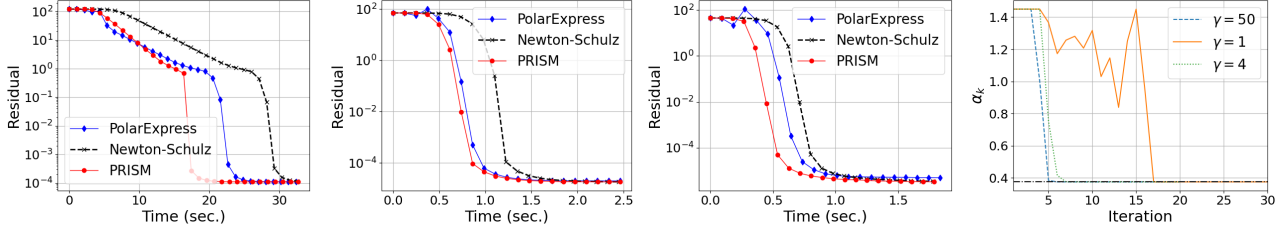


Figure 3. Convergence of degree-5 polynomial methods for orthogonalizing a Gaussian random matrix $\mathbf{A} \in \mathbb{R}^{n \times m}$ with varying aspect ratio $\gamma = n/m$. The figures from left to right show the Frobenius norm error $\|\mathbf{I} - \mathbf{X}_k^T \mathbf{X}_k\|_F$ for $\gamma = 1, 4, 50$, respectively. The last figure on the right shows the α_k 's computed by (4) in PRISM for different aspect ratios at each iteration.

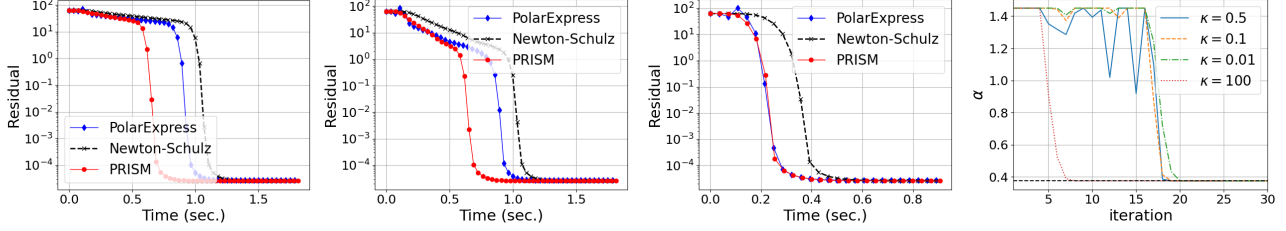


Figure 4. Convergence of degree-5 polynomial methods for orthogonalizing random matrices generated by HTMP (Hodgkinson et al., 2025) with different parameter κ . Smaller κ indicates a heavier tail in the spectra. The figures from left to right show the Frobenius norm error $\|\mathbf{I} - \mathbf{X}_k^T \mathbf{X}_k\|_F$ for $\kappa = 0.1, 0.5, 100$, respectively. The rightmost figure shows the α_k 's computed by (4) in PRISM.

a degree-4 polynomial in α whose coefficients are linear functions of $\text{tr}(\mathbf{S}_k \mathbf{R}_k^i \mathbf{S}_k^T)$ for $1 \leq i \leq 4d + 2$. Therefore, computing these coefficients now requires $O(n^2 p)$ time, as opposed to $O(n^3)$ in the previous case. This means that computing $\tilde{\alpha}_k$ takes $O(n^2 p)$ time in total, which can be much less than the $O(n^3)$ complexity of one iteration of Newton-Schulz. There are many plausible choices for the sketch matrix $\mathbf{S}_k$, and here simple random Gaussian matrices appear to be sufficient.

One might ask whether we will lose the convergence speed when we replace the exact minimization in (3) with the approximate minimization in (4). For $d = 1$, Theorem 4.2 says that the worst-case convergence rate is essentially the same when $p = O(\log n)$. The proof uses Johnson-Lindenstrauss property of OSE and the strong convexity of $m(\alpha)$ to bound the distance between α_k^* from (3) and $\tilde{\alpha}_k$ from (4), and then shows that the resulting polynomial $g_d(\xi; \tilde{\alpha}_k)$ still induces a similar quadratic convergence property as $g_d(\xi; \alpha_k^*)$. We leave the proof to Appendix C.3. A similar line of arguments should generalize to the case $d \geq 2$. Empirically, for both $d = 1$ and $d = 2$, we observed that the dimension p can be as small as 5 and still the sequence $\mathbf{X}_k$ converges as fast as if α_k^* were computed in (3) without sketching.

Theorem 4.2. *Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be such that $0 < \|\mathbf{A}\|_2 \leq 1$ and $\mathbf{A}^2$ is symmetric. Let $\mathbf{S}_k \in \mathbb{R}^{p \times n}$ be random matrices consisting of i.i.d Gaussian entries $[\mathbf{S}_k]_{i,j} \sim \mathcal{N}(1, 1/p)$ and $p \geq 48(\log n + \log(1/\delta) + \log k + 27.6)$. Let $\mathbf{X}_0 = \mathbf{A}$ and consider the sequence of matrices $\mathbf{X}_1, \mathbf{X}_2, \dots$ generated by (2) where α_k^* is determined by (4), with $d = 1, \ell = 1/2$*

and $u = 1$. With probability at least $1 - \delta$, we have that $\mathbf{X}_k \rightarrow \text{sign}(\mathbf{A})$ and $\|\mathbf{I} - \mathbf{X}_k^2\|_2 \leq \|\mathbf{I} - \mathbf{A}^2\|_2^{2^{k-3}}$.

Remark. Using a recent result on the subspace embedding property of rescaled Gaussian sketch matrices, e.g., see Proposition B.3 in Balabanov et al. (2026), the $\log n$ factor in the sketch dimension p may be replaced by a constant, and consequently a constant sketch dimension $p = O(1)$ may already be sufficient to obtain the worst-case guarantee in Theorem 4.2. This partially explains why empirically a sketch dimension p as small as 5 works in our case. The overall convergence behavior in practice is surprisingly stable with respect to small sketch matrices.

5. PRISM-based Computation of Square Roots, Orthogonalization and Others

The following results of Higham et al. (2004) and Higham (1997) imply that what we derived for matrix sign computation readily extend to square root and orthogonalization.

Theorem 5.1 ((Higham, 1997)). *Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ have no eigenvalues on $\mathbb{R}_-$. Consider any iteration of the form $\mathbf{X}_{k+1} = \mathbf{X}_k h(\mathbf{X}_k^2)$ that converges to $\text{sign}(\mathbf{X}_0)$ for $\mathbf{X}_0 = \begin{bmatrix} 0 & \mathbf{A} \\ \mathbf{I} & 0 \end{bmatrix}$ with order of convergence q . Then in the coupled iteration $\mathbf{X}_{k+1} = \mathbf{X}_k h(\mathbf{Y}_k \mathbf{X}_k)$, $\mathbf{Y}_{k+1} = h(\mathbf{Y}_k \mathbf{X}_k) \mathbf{Y}_k$, with $\mathbf{X}_0 = \mathbf{A}$ and $\mathbf{Y}_0 = \mathbf{I}$, we have $\mathbf{X}_k \rightarrow \mathbf{A}^{1/2}$ and $\mathbf{Y}_k \rightarrow \mathbf{A}^{-1/2}$, both with order of convergence q .*

Theorem 5.2 ((Higham et al., 2004)). *Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m \geq n$ be of rank n and have SVD $\mathbf{A} = \mathbf{U} \Sigma \mathbf{V}^T$.*

Consider any iteration of the form $\mathbf{X}_{k+1} = \mathbf{X}_k h(\mathbf{X}_k^2)$ that converges to $\text{sign}(\mathbf{X}_0)$ for $\mathbf{X}_0 = (\mathbf{A}^T \mathbf{A})^{1/2}$ with order of convergence q . Then $\mathbf{X}_{k+1} = \mathbf{X}_k h(\mathbf{X}_k^T \mathbf{X}_k)$ with $\mathbf{X}_0 = \mathbf{A}$ converges to $\mathbf{U}\mathbf{V}^T$ with order of convergence q .

Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be a symmetric matrix with positive real eigenvalues, and let $\mathbf{X}_0 = \begin{bmatrix} 0 & \mathbf{A} \\ \mathbf{I} & 0 \end{bmatrix}$. Then $\mathbf{X}_0^2$ is symmetric, and thus we can use PRISM to accelerate Newton-Schulz for $\text{sign}(\mathbf{X}_0)$. Theorem 5.1 guarantees the following iteration converges to the square roots of $\mathbf{A}$:

$$\begin{aligned} \mathbf{X}_0 &= \mathbf{A}, \mathbf{Y}_0 = \mathbf{I}, \mathbf{R}_k = \mathbf{I} - \mathbf{X}_k \mathbf{Y}_k \\ \mathbf{X}_{k+1} &= \mathbf{X}_k g_d(\mathbf{R}_k; \tilde{\alpha}_k), \mathbf{Y}_{k+1} = g_d(\mathbf{R}_k; \tilde{\alpha}_k) \mathbf{Y}_k, \end{aligned}$$

where $g_d(\xi; \alpha) = f_{d-1}(\xi) + \alpha \xi^d$ and f_d is the d -order Taylor polynomial of $f(\xi) = (1 - \xi)^{-1/2}$, and $\tilde{\alpha}_k$ is computed according to (4). For $d = 1$, Theorem 4.2 and Theorem 5.1 imply that it has quadratic convergence in the worst case. Similarly, one may obtain PRISM-accelerated Newton-Schulz for orthogonalization. Table 1 shows these methods for $d \in \{1, 2\}$ in explicit forms, which correspond to accelerated variants of the 3rd- and 5th-order Newton-Schulz iterations, respectively. See Appendix A.1 for details on these algorithms are derived. Pseudocode for each of these PRISM-based algorithms is provided in Appendix B.

Adaptive polynomial acceleration of other algorithms, such as those provided in Table 1, can be obtained analogously to how Part II of PRISM applies to accelerate matrix sign computation. In Appendix A, we provide detailed derivations for every algorithm from Table 1 along with explicit formulas on how to compute α_k in each case.

6. Experiments

6.1. Empirical Evaluation of Fast Convergence

We empirically test the accelerated convergence of PRISM-based Newton-Schulz (5th-order, cf. Table 1, and also Algorithm 2 in Appendix B) for computing the polar factor of matrix $\mathbf{A} \in \mathbb{R}^{n \times m}$, comparing it with the classical Newton-Schulz and PolarExpress (Amsel et al., 2026). Since PRISM has an additional overhead to dynamically compute α_k , to make the comparison fair, we measure the wall-clock time used by each algorithm. Convergence with respect to the number of iterations is shown in Appendix F. In Figure 3, we use standard Gaussian random matrices with different aspect ratios $\gamma = n/m$ and compare the convergence of the Frobenius norm error for 5th-order Newton-Schulz, PolarExpress, and PRISM. In Figure 4, we carry out the same experiment for matrices with heavy-tailed spectra. Many recent works (Martin & Mahoney, 2019; 2020; Wang et al., 2023) have observed that the weight and kernel matrices in well-trained neural networks have heavy-tailed spectra. The spectra of gradient matrices in well-trained models often inherit heavy tails. We follow Hodgkinson et al. (2025)

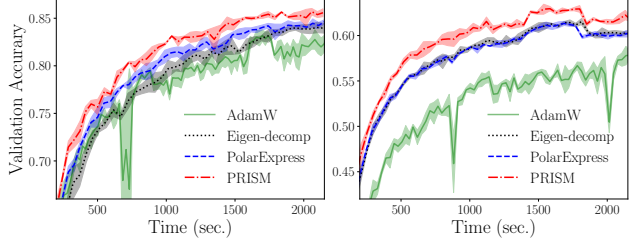


Figure 5. Improvement to the Shampoo optimizer in terms of training speed. We compare three methods to compute the inverse root preconditioner inside Shampoo. Left: ResNet-20 on CIFAR10. Right: ResNet-32 on CIFAR100.

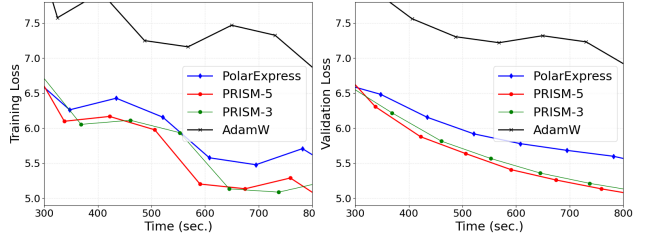


Figure 6. Improvement of Muon in terms of training and validation losses for the GPT-2 model. The final validation losses for PolarExpress, PRISM with degree-5 (PRISM-5) and degree-3 (PRISM-3) polynomials, and AdamW are 5.4523, 5.0251, 4.9886, and 6.8689.

to generate high-temperature Marchenko-Pastur (HTMP) random matrices to mimic the heavy-tailed gradient matrices in well-trained neural networks as the input matrix. In Figures 3 and 4 we also plot the evolutions of the coefficient α_k found by PRISM, which exhibit very different trends for different input matrices. Automatically adapting to the input spectra allows PRISM to converge the fastest in our experiments. For additional experiments on square roots, see Appendix F.

6.2. Applications to Neural Network Training

We integrate PRISM into neural network optimizers that require frequent computation of matrix functions and compare performance with existing methods. We carry out experiments using the Shampoo (Shi et al., 2023) and Muon (Jordan et al., 2024; Amsel et al., 2026) optimizers.

Shampoo is a preconditioned stochastic gradient method that generalizes AdaGrad (Duchi et al., 2011). For a given weight matrix $\mathbf{W}_t$ and gradient $\mathbf{G}_t$, the update step with learning rate η is $\mathbf{W}_{t+1} = \mathbf{W}_t - \eta \mathbf{L}_t^{-1/p} \mathbf{G}_t \mathbf{R}_t^{-1/p}$, where $\mathbf{L}_t, \mathbf{R}_t$ are two preconditioners maintained by Shampoo. Recent work recommended using $p = 2$ (Shi et al., 2023; Morwani et al., 2025) and this is what we use in our experiment. Previous implementations use eigen-decomposition to compute inverse roots $\mathbf{L}_t^{-1/2}$ and $\mathbf{R}_t^{-1/2}$. We replace this part with PRISM (accelerated 5th-order Newton-Schulz,

cf. Table 1, and also Algorithm 4 in Appendix B) and PolarExpress and compare their performance with eigen-decomposition.³ We train slightly larger variants of ResNet-20 and ResNet-32 (He et al., 2016) for the CIFAR10 and CIFAR100 datasets. We run 5 iterations for both PolarExpress and PRISM (accelerated 5-th order Newton-Schulz). The validation accuracy throughout the first 50 epochs is shown in Figure 5. The ranking stays the same when we keep training longer.

The Muon optimizer belongs to the family of spectral descent algorithms (Carlson et al., 2015; Riabinin et al., 2025; Su, 2025; Davis & Drusvyatskiy, 2025). It gained popularity as an alternative for training large language models (Liu et al., 2025; Shah et al., 2025; Wen et al., 2025). In Figure 6, we train a GPT-2 model from random initialization with 10 layers, 16 attention heads, and an embedding dimension of 1024, using 200M tokens from the FineWeb dataset. We implement the polar decomposition of gradient matrices inside Muon with PolarExpress, PRISM-based Newton-Schulz with degree-3 and degree-5 polynomials. Additional experimental details can be found in Appendix E.

Table 2. NanoGPT trained by PRISM-5 with various iterations

Iterations	1	3	5	7	9
Validation loss	4.926	4.403	4.015	3.890	3.885
Time (min)	26.5	29.2	32.3	35.3	38.2
Avg LMO error	26.87	23.20	8.97	1.55	0.56

More iterations of PRISM at each step of training lower the residual error and the weight updates conform more strictly to exact orthogonalization. However, this also increases the time spent per training step. Table 2 shows results of training NanoGPT with 124M training parameters on FineWeb for 2200 steps with varying PRISM-5 iterations at each step. Here, Avg LMO error refers to mean Frobenius residual error of polar decomposition of the gradient matrices over layers at final step. Observe that a lower residual directly correlates with a lower validation loss but a strict trade-off exists (Shulgin et al., 2025): moving from 3 to 5 iterations increases total training time by 20% while yielding only marginal improvements in validation loss.

7. Conclusion

PRISM frames a wide range of classical iterations under a single meta-algorithmic template (Part I), then accelerates them (Part II) by dynamically fitting polynomial updates to the evolving spectrum of the current residual, without requiring a priori spectral bounds or distributional assumptions on singular values.

³Using Theorem 5.1, PolarExpress (Amsel et al., 2026) can be used in a coupled form to compute matrix (inverse) square root faster than the classical Newton-Schulz.

Empirically, PRISM consistently delivers robust speedups across spectra that are common in ML practice, including Marchenko–Pastur-like and heavy-tailed regimes. As a result, PRISM effectively accelerates training when integrated into methods such as Shampoo and Muon. Overall, this provides a practical and general route to instance-adaptive, GPU-friendly matrix function computation, turning spectral adaptivity—previously a source of tuning burden—into a reliable algorithmic primitive for large-scale computations.

Acknowledgements

MWM would like to acknowledge the support of the Defense Advanced Research Projects Agency (DARPA) under Contract No. HR0011-25-2-0011. SY would also like to acknowledge the support of the Natural Sciences and Engineering Research Council of Canada (NSERC). NBE would also like to acknowledge the support from the U.S. Department of Energy, Office of Advanced Scientific Computing Research, Scientific Discovery through Advanced Computing (SciDAC) program, under Contract Number DE-AC02-05CH11231 at Berkeley Lab. The views, opinions, and/or findings expressed are those of the authors and should not be interpreted as representing the official views or policies of the Department of Defense or the U.S. Government.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Absil, P.-A., Mahony, R., and Sepulchre, R. *Optimization algorithms on matrix manifolds*. Princeton University Press, 2008.
- Ahn, K., Xu, B., Abreu, N., and Langford, J. Dion: Distributed orthonormalized updates. *arXiv preprint arXiv:2504.05295*, 2025. URL <https://arxiv.org/abs/2504.05295>.
- Amsel, N., Persson, D., Musco, C., and Gower, R. M. The polar express: Optimal matrix sign methods and their application to the muon algorithm. In *International Conference on Learning Representations (ICLR)*, 2026. URL <https://openreview.net/forum?id=yRtgZ1K8hO>.
- Arisaka, S. and Li, Q. Principled acceleration of iterative numerical methods using machine learning. In *Inter-*

- national Conference on Machine Learning (ICML)*, pp. 1041–1059. PMLR, 2023.
- Avron, H. and Toledo, S. Randomized algorithms for estimating the trace of an implicit symmetric positive semi-definite matrix. *Journal of the ACM (JACM)*, 58(2):1–34, 2011.
- Balabanov, O. and Nouy, A. Randomized linear algebra for model reduction. part i: Galerkin methods and error estimation. *Advances in Computational Mathematics*, 45: 2969–3019, 2019.
- Balabanov, O., Nouy, A., and Pasco, A. Random sketching of operators with application to learning preconditioners, 2026. URL <https://arxiv.org/abs/2104.12177>.
- Carlson, D. E., Collins, E., Hsieh, Y.-P., Carin, L., and Cevher, V. Preconditioned spectral descent for deep learning. In *Advances in Neural Information Processing Systems (NeurIPS)*. MIT Press, 2015.
- Castro, M. J., Gallardo, J. M., and Marquina, A. Approximate osher–solomon schemes for hyperbolic systems. *Applied Mathematics and Computation*, 272:347–368, 2016.
- Chen, J., Anitescu, M., and Saad, Y. Computing $f(A)b$ via least squares polynomial approximations. *SIAM Journal on Scientific Computing*, 33(1):195–222, 2011. doi: 10.1137/090778250. URL <https://doi.org/10.1137/090778250>.
- Chen, Y., Fan, J., Ma, C., and Yan, Y. Inference and uncertainty quantification for noisy matrix completion. *Proceedings of the National Academy of Sciences*, 116(46):22931–22937, 2019. doi: 10.1073/pnas.1910053116. URL <https://www.pnas.org/doi/abs/10.1073/pnas.1910053116>.
- Cheng, S. H., Higham, N. J., Kenney, C. S., and Laub, A. J. Approximating the logarithm of a matrix to specified accuracy. *SIAM Journal on Matrix Analysis and Applications*, 22(4):1112–1125, 2001. doi: 10.1137/S0895479899364015. URL <https://doi.org/10.1137/S0895479899364015>.
- Cortinovis, A. and Kressner, D. On randomized trace estimates for indefinite matrices with an application to determinants. *Foundations of Computational Mathematics*, 22(3):875–903, 2022.
- Davis, D. and Drusvyatskiy, D. When do spectral gradient updates help in deep learning? *arXiv preprint arXiv:2512.04299*, 2025. URL <https://arxiv.org/abs/2512.04299>.
- Denman, E. D. and Beavers, A. N. The matrix sign function and computations in systems. *Appl. Math. Comput.*, 2(1): 63–94, January 1976. ISSN 0096-3003. doi: 10.1016/0096-3003(76)90020-5. URL [https://doi.org/10.1016/0096-3003\(76\)90020-5](https://doi.org/10.1016/0096-3003(76)90020-5).
- Drineas, P. and Mahoney, M. W. Lectures on randomized numerical linear algebra. In Mahoney, M. W., Duchi, J. C., and Gilbert, A. C. (eds.), *The Mathematics of Data*, IAS/Park City Mathematics Series, pp. 1–48. AMS/IAS/SIAM, 2018.
- Drineas, P., Mahoney, M. W., and Muthukrishnan, S. Sampling algorithms for ℓ_2 regression and applications. In *Proceedings of the 17th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 1127–1136, 2006.
- Drineas, P., Mahoney, M. W., Muthukrishnan, S., and Sarlós, T. Faster least squares approximation. *Numerische Mathematik*, 117(2):219–249, 2011.
- Duchi, J., Hazan, E., and Singer, Y. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research (JMLR)*, 12(61):2121–2159, 2011. URL <http://jmlr.org/papers/v12/duchilla.html>.
- Fan, L., Shuman, D. I., Ubaru, S., and Saad, Y. Spectrum-adapted polynomial approximation for matrix functions. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 4933–4937, 2019. doi: 10.1109/ICASSP.2019.8683179.
- Fan, T., Shuman, D. I., Ubaru, S., and Saad, Y. Spectrum-adapted polynomial approximation for matrix functions with applications in graph signal processing. *Algorithms*, 13(11):295, 2020.
- Grishina, E., Smirnov, M., and Rakhuba, M. Accelerating newton–schulz iteration for orthogonalization via chebyshev-type polynomials. *arXiv preprint arXiv:2506.10935*, 2025. doi: 10.48550/arXiv.2506.10935. URL <https://arxiv.org/abs/2506.10935>.
- Gupta, V., Koren, T., and Singer, Y. Shampoo: Preconditioned stochastic tensor optimization. In *International Conference on Machine Learning (ICML)*, pp. 1842–1850, 2018.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016. doi: 10.1109/CVPR.2016.90.
- Higham, N. J. Stable iterations for the matrix square root. *Numerical Algorithms*, 15(2):227–242, September 1997. doi: 10.1023/A:1019150005407. URL <https://doi.org/10.1023/A:1019150005407>.

- Higham, N. J., Mackey, D. S., Mackey, N., and Tisseur, F. Computing the polar decomposition and the matrix sign decomposition in matrix groups. *SIAM journal on matrix analysis and applications*, 25(4):1178–1192, 2004.
- Higham, N. J., Mackey, D. S., Mackey, N., and Tisseur, F. Functions preserving matrix groups and iterations for the matrix square root. *SIAM Journal on Matrix Analysis and Applications*, 26(3):849–877, 2005.
- Hodgkinson, L., Wang, Z., and Mahoney, M. W. Models of heavy-tailed mechanistic universality. In *Forty-second International Conference on Machine Learning (ICML)*, 2025. URL <https://openreview.net/forum?id=r4XsTcuiqc>.
- Huang, F., Luo, Y., and Chen, S. Limuon: Light and fast muon optimizer for large models. *arXiv preprint arXiv:2509.14562*, 2025.
- Huang, L., Yang, D., Lang, B., and Deng, J. Decorrelated batch normalization. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 791–800, 2018.
- Janati, H., Muzellec, B., Peyré, G., and Cuturi, M. Entropic optimal transport between unbalanced gaussian measures has a closed form. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Jordan, K., Jin, Y., Boza, V., You, J., Cesista, F., Newhouse, L., and Bernstein, J. Muon: An optimizer for hidden layers in neural networks, 2024. URL <https://kellerjordan.github.io/posts/muon/>.
- Kenney, C. and Laub, A. J. Rational iterative methods for the matrix sign function. *SIAM Journal on Matrix Analysis and Applications*, 12(2):273–291, 1991. doi: 10.1137/0612020. URL <https://doi.org/10.1137/0612020>.
- Kim, S., Dwaraknath, R. V., Pilanci, M., et al. Matrl: Provably generalizable iterative algorithm discovery via monte-carlo tree search. *arXiv preprint arXiv:2507.03833*, 2025.
- Li, P., Xie, J., Wang, Q., and Gao, Z. Towards faster training of global covariance pooling networks by iterative matrix square root normalization. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 947–955, 2018.
- Li, Y., Fang, C., Yang, J., Wang, Z., Lu, X., and Yang, M.-H. Universal style transfer via feature transforms. *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- Lin, L., Lu, J., Car, R., and E, W. Multipole representation of the fermi operator with application to the electronic structure analysis of metallic systems. *Physical Review B—Condensed Matter and Materials Physics*, 79(11): 115133, 2009a.
- Lin, L., Lu, J., Ying, L., Car, R., and E, W. Fast algorithm for extracting the diagonal of the inverse matrix with application to the electronic structure analysis of metallic systems. *Communications in Mathematical Sciences*, 7(3):755 – 777, 2009b.
- Liu, J., Su, J., Yao, X., Jiang, Z., Lai, G., Du, Y., Qin, Y., Xu, W., Lu, E., Yan, J., Chen, Y., Zheng, H., Liu, Y., Liu, S., Yin, B., He, W., Zhu, H., Wang, Y., Wang, J., Dong, M., Zhang, Z., Kang, Y., Zhang, H., Xu, X., Zhang, Y., Wu, Y., Zhou, X., and Yang, Z. Muon is scalable for llm training. *arXiv preprint arXiv:2502.16982*, 2025. URL <https://arxiv.org/abs/2502.16982>.
- Mahoney, M. W. *Randomized algorithms for matrices and data*. Foundations and Trends in Machine Learning. NOW Publishers, Boston, 2011.
- Mallasto, A. and Feragen, A. Learning from uncertain curves: The 2-wasserstein metric for gaussian processes. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- Markidis, S., Chien, S. W. D., Laure, E., Peng, I. B., and Vetter, J. S. Nvidia tensor core programmability, performance & precision. In *IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2018. doi: 10.1109/IPDPSW.2018.00091.
- Martin, C. H. and Mahoney, M. W. Traditional and heavy tailed self regularization in neural network models. In *International Conference on Machine Learning (ICML)*, pp. 4284–4293, 2019.
- Martin, C. H. and Mahoney, M. W. Heavy-tailed universality predicts trends in test accuracies for very large pre-trained deep neural networks. In *Proceedings of the SIAM International Conference on Data Mining*, pp. 505–513, 2020.
- Martin, C. H. and Mahoney, M. W. Implicit self-regularization in deep neural networks: evidence from random matrix theory and implications for learning. *Journal of Machine Learning Research (JMLR)*, 22(1), January 2021. ISSN 1532-4435.
- Meyer, R. A., Musco, C., Musco, C., and Woodruff, D. P. Hutch++: Optimal stochastic trace estimation. In *Symposium on Simplicity in Algorithms (SOSA)*, pp. 142–155, 2021.

- Minh, H. Q. Alpha procrustes metrics between positive definite operators: A unifying formulation for the bures-wasserstein and log-euclidean/log-hilbert-schmidt metrics. *Linear Algebra and its Applications*, 636:25–68, 2022. ISSN 0024-3795. doi: <https://doi.org/10.1016/j.laa.2021.11.011>.
- Morwani, D., Shapira, I., Vyas, N., Eran Malach, Kakade, S. M., and Janson, L. A new perspective on shampoo’s preconditioner. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL <https://openreview.net/forum?id=c6zI3Cp8c6>.
- Murray, R., Demmel, J., Mahoney, M. W., Erichson, N. B., Melnichenko, M., Malik, O. A., Grigori, L., Luszczek, P., Dereziński, M., Lopes, M. E., Liang, T., Luo, H., and Dongarra, J. Randomized Numerical Linear Algebra – a perspective on the field with an eye to software. Technical Report Preprint: arXiv:2302.11474v2, 2023.
- Nakatsukasa, Y. and Freund, R. W. Computing fundamental matrix decompositions accurately via the matrix sign function in two iterations: The power of zolotarev’s functions. *SIAM Review*, 58(3):461–493, 2016.
- Nakatsukasa, Y. and Higham, N. J. Backward stability of iterations for computing the polar decomposition. *SIAM Journal on Matrix Analysis and Applications*, 33(2):460–479, 2012.
- Ndjinga, M. Computing the matrix sign and absolute value functions. *Comptes Rendus Mathématique*, 346(1-2):119–124, 2008.
- Nelson, J. and Nguyễn, H. L. Osnap: Faster numerical linear algebra algorithms via sparser subspace embeddings. In *IEEE Symposium on Foundations of Computer Science (FOCS)*, 2013.
- Pleiss, G., Jankowiak, M., Eriksson, D., Damle, A., and Gardner, J. Fast matrix square roots with applications to gaussian processes and bayesian optimization. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 2020.
- Refael, Y., Smorodinsky, G., Tirer, T., and Lindenbaum, O. Sumo: Subspace-aware moment-orthogonalization for accelerating memory-efficient llm training. *arXiv preprint arXiv:2505.24749*, 2025.
- Riabini, A., Shulgin, E., Gruntkowska, K., and Richtárik, P. Gluon: Making muon & scion great again! (bridging theory and practice of lmo-based optimizers for llms). *arXiv preprint arXiv:2505.13416*, 2025. URL <https://arxiv.org/abs/2505.13416>.
- Roosta-Khorasani, F. and Ascher, U. Improved bounds on sample size for implicit matrix trace estimators. *Foundations of Computational Mathematics*, 15(5):1187–1212, 2015.
- Sarlós, T. Improved approximation algorithms for large matrices via random projections. In *IEEE Symposium on Foundations of Computer Science (FOCS)*, pp. 143–152, 2006.
- Shah, I., Polloreno, A. M., Stratos, K., Monk, P., Chaluvareju, A., Hojel, A., Ma, A., Thomas, A., Tanwer, A., Shah, D. J., et al. Practical efficiency of muon for pre-training. *arXiv preprint arXiv:2505.02222*, 2025. URL <https://arxiv.org/abs/2505.02222>. Essential AI.
- Shi, H.-J. M., Lee, T.-H., Iwasaki, S., Gallego-Posada, J., Li, Z., Rangadurai, K., Mudigere, D., and Rabbat, M. A distributed data-parallel pytorch implementation of the distributed shampoo optimizer for training neural networks at-scale, 2023. URL <https://arxiv.org/abs/2309.06497>.
- Shulgin, E., AlRashed, S., Orabona, F., and Richtárik, P. Beyond the ideal: Analyzing the inexact muon update. *arXiv preprint arXiv:2510.19933*, 2025.
- Song, Y., Sebe, N., and Wang, W. Why approximate matrix square root outperforms accurate svd in global covariance pooling? In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE Computer Society, 2021. doi: 10.1109/ICCV48922.2021.00115.
- Song, Y., Sebe, N., and Wang, W. Fast differentiable matrix square root and inverse square root. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 45(6):7367–7380, 2023. doi: 10.1109/TPAMI.2022.3216339.
- Su, W. Isotropic curvature model for understanding deep learning optimization: Is gradient orthogonalization optimal? *arXiv preprint arXiv:2511.00674*, 2025.
- Ubaru, S., Chen, J., and Saad, Y. Fast estimation of $\text{tr}(f(A))$ via stochastic lanczos quadrature. *SIAM Journal on Matrix Analysis and Applications*, 38(4):1075–1099, 2017.
- Volkov, V. and Demmel, J. W. Benchmarking gpus to tune dense linear algebra. In *Proceedings of the ACM/IEEE Conference on Supercomputing (SC)*, 2008. doi: 10.1109/SC.2008.5214359.
- Wang, Q., Xie, J., Zuo, W., Zhang, L., and Li, P. Deep cnns meet global covariance pooling: Better representation and generalization. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 43(8):2582–2597, 2021. doi: 10.1109/TPAMI.2020.2974833.

- Wang, Z., Engel, A., Sarwate, A. D., Dumitriu, I., and Chiang, T. Spectral evolution and invariance in linear-width neural networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 36:20695–20728, 2023.
- Wen, K., Hall, D., Ma, T., and Liang, P. Fantastic pretraining optimizers and where to find them. *arXiv preprint arXiv:2509.02046*, 2025.
- Woodruff, D. P. Sketching as a tool for numerical linear algebra. *Foundations and Trends® in Theoretical Computer Science*, 10(1–2):1–157, 2014.
- Yan, D., Wang, W., and Chu, X. Demystifying tensor cores to optimize half-precision matrix multiply. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2020. doi: 10.1109/IPDPS47924.2020.00071.
- Yao, Z., Gholami, A., Keutzer, K., and Mahoney, M. W. Pyhessian: Neural networks through the lens of the hessian. In *IEEE International Conference on Big Data*, pp. 581–590. IEEE, 2020.
- Yao, Z., Gholami, A., Shen, S., Mustafa, M., Keutzer, K., and Mahoney, M. Adahessian: An adaptive second order optimizer for machine learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021.

Appendices

Contents

A	Deriving PRISM-based Algorithms	15
A.1	Newton-Schulz iteration for matrix sign, square roots and polar decomposition	15
A.2	DB Newton iteration for matrix square roots	16
A.3	Inverse Newton iteration for inverse p -th root	18
A.4	Chebyshev’s iteration for inverse	19
B	Pseudocode of Newton-Schulz Variants for Polar Decomposition and Square Roots	20
B.1	Algorithm 1: PRISM Newton-Schulz (3rd order) for polar decomposition	20
B.2	Algorithm 2: PRISM Newton-Schulz (5th order) for polar decomposition	21
B.3	Algorithm 3: PRISM Newton-Schulz (3rd order) for square roots	21
B.4	Algorithm 4: PRISM Newton-Schulz (5th order) for square roots	22
C	Proof of Theorems in the Main Text	23
C.1	Technical lemma	23
C.2	Proof of Theorem 4.1	30
C.3	Proof of Theorem 4.2	31
D	On Coefficient Bound $[\ell, u]$ for Polynomial Optimization in Higher-Order Newton-Schulz Variants	33
D.1	Choosing $[\ell, u]$ for PRISM-based 5th-order Newton-Schulz and beyond	34
E	Details of Numerical Experiments	34
F	Additional Empirical Results	35

A. Deriving PRISM-based Algorithms

A.1. Newton-Schulz iteration for matrix sign, square roots and polar decomposition

For completeness, we first apply Part I of the PRISM meta-algorithm to derive the Newton-Schulz iteration for computing the matrix sign function, and then we apply Part II to accelerate its convergence. Afterwards, we will invoke Theorem 5.1 and Theorem 5.2 to obtain PRISM-accelerated Newton-Schulz iteration for square roots and polar decomposition, respectively. For the computation of the matrix sign function, we will assume that the input matrix $\mathbf{A}$ is such that $\mathbf{A}^2$ is symmetric and $\|\mathbf{A}\|_2 \leq 1$.

Let $x \neq 0$ be such that $\text{sign}(x) = \text{sign}(a)$. Then we can write

$$\text{sign}(a) = \text{sign}(x) = x(x^2)^{-1/2} = x(1 - \xi)^{-1/2} = xf(\xi),$$

where $f(\xi) = (1 - \xi)^{-1/2}$, and where $\xi = 1 - x^2$ measures how close x is to $\text{sign}(x) = \text{sign}(a)$. Therefore, the problem of approximating $\text{sign}(a)$ leads to that of approximating $f(\xi)$. Using the d -th order Taylor polynomial $f_d(\xi)$ around $\xi = 0$, we obtain an iterative procedure

$$x_{k+1} = x_k f_d(\xi(x_k)).$$

The matrix version,

$$\mathbf{X}_0 = \mathbf{A}, \mathbf{R}_k = \mathbf{I} - \mathbf{X}_k^2, \mathbf{X}_{k+1} = \mathbf{X}_k f_d(\mathbf{R}_k),$$

is the (generalized) Newton-Schulz iteration for matrix sign (Higham et al., 2005). To accelerate the convergence, Part II of PRISM defines

$$g_d(\xi; \alpha) = f_{d-1}(\xi) + \alpha \xi^d$$

and the following iteration

$$\begin{aligned} \mathbf{X}_0 &= \mathbf{A}, \mathbf{R}_k = \mathbf{I} - \mathbf{X}_k, \mathbf{X}_{k+1} = \mathbf{X}_k g_d(\mathbf{X}_k; \alpha_k), \\ \text{where } \alpha_k &= \arg \min_{\alpha \in [\ell, u]} \left\| \mathbf{S}_k \left(\mathbf{I} - \mathbf{X}_k^2 g_d(\mathbf{R}_k; \alpha)^2 \right) \right\|_F^2, \end{aligned}$$

and $\mathbf{S}_k \in \mathbb{R}^{p \times n}$ with $m \ll n$ is a sketch matrix, e.g., consisting of i.i.d Gaussian entries. For $d = 1$, Theorem 4.2 guarantees quadratic convergence if we set $[\ell, u] = [1/2, 1]$ in the computation of α_k . For $d = 2$, empirically we find that $[\ell, u] = [3/8, 29/20]$ always leads to fast convergence.

Denote the optimization objective function in the definition of α_k as

$$m(\alpha) = \left\| \mathbf{S}_k \left(\mathbf{I} - \mathbf{X}_k^2 g_d(\mathbf{R}_k; \alpha)^2 \right) \right\|_F^2.$$

Then $m(\alpha)$ is a degree-4 polynomial with respect to α , that is

$$m(\alpha) = c_0 + c_1 \alpha + c_2 \alpha^2 + c_3 \alpha^3 + c_4 \alpha^4.$$

The coefficients c_0, c_1, c_2, c_3, c_4 depend on d . For $d = 1$ we have $g_1(\xi; \alpha) = 1 + \alpha \xi$, a simple calculation yields that

$$\begin{aligned} c_1 &= 4 \text{tr}(\mathbf{S}_k \mathbf{R}_k^3 \mathbf{S}_k^T) - 4 \text{tr}(\mathbf{S}_k \mathbf{R}_k^2 \mathbf{S}_k^T), \\ c_2 &= 6 \text{tr}(\mathbf{S}_k \mathbf{R}_k^4 \mathbf{S}_k^T) - 10 \text{tr}(\mathbf{S}_k \mathbf{R}_k^3 \mathbf{S}_k^T) + 4 \text{tr}(\mathbf{S}_k \mathbf{R}_k^2 \mathbf{S}_k^T), \\ c_3 &= 4 \text{tr}(\mathbf{S}_k \mathbf{R}_k^5 \mathbf{S}_k^T) - 8 \text{tr}(\mathbf{S}_k \mathbf{R}_k^4 \mathbf{S}_k^T) + 4 \text{tr}(\mathbf{S}_k \mathbf{R}_k^3 \mathbf{S}_k^T), \\ c_4 &= \text{tr}(\mathbf{S}_k \mathbf{R}_k^6 \mathbf{S}_k^T) - 2 \text{tr}(\mathbf{S}_k \mathbf{R}_k^5 \mathbf{S}_k^T) + \text{tr}(\mathbf{S}_k \mathbf{R}_k^4 \mathbf{S}_k^T). \end{aligned}$$

For $d = 2$ and $g_2(\xi; \alpha) = 1 + \frac{1}{2}\xi + \alpha \xi^2$, we have

$$\begin{aligned} c_1 &= \frac{1}{2} \text{tr}(\mathbf{S}_k \mathbf{R}_k^7 \mathbf{S}_k^T) + 2 \text{tr}(\mathbf{S}_k \mathbf{R}_k^6 \mathbf{S}_k^T) + \frac{1}{2} \text{tr}(\mathbf{S}_k \mathbf{R}_k^5 \mathbf{S}_k^T) - 3 \text{tr}(\mathbf{S}_k \mathbf{R}_k^4 \mathbf{S}_k^T), \\ c_2 &= \frac{3}{2} \text{tr}(\mathbf{S}_k \mathbf{R}_k^8 \mathbf{S}_k^T) + 3 \text{tr}(\mathbf{S}_k \mathbf{R}_k^7 \mathbf{S}_k^T) - \frac{9}{2} \text{tr}(\mathbf{S}_k \mathbf{R}_k^6 \mathbf{S}_k^T) - 4 \text{tr}(\mathbf{S}_k \mathbf{R}_k^5 \mathbf{S}_k^T) + 4 \text{tr}(\mathbf{S}_k \mathbf{R}_k^4 \mathbf{S}_k^T), \\ c_3 &= 2 \text{tr}(\mathbf{S}_k \mathbf{R}_k^9 \mathbf{S}_k^T) - 6 \text{tr}(\mathbf{S}_k \mathbf{R}_k^7 \mathbf{S}_k^T) + 4 \text{tr}(\mathbf{S}_k \mathbf{R}_k^6 \mathbf{S}_k^T), \\ c_4 &= \text{tr}(\mathbf{S}_k \mathbf{R}_k^{10} \mathbf{S}_k^T) - 2 \text{tr}(\mathbf{S}_k \mathbf{R}_k^9 \mathbf{S}_k^T) + \text{tr}(\mathbf{S}_k \mathbf{R}_k^8 \mathbf{S}_k^T). \end{aligned}$$

For general $d \geq 1$, computing these coefficients requires access to the diagonal entries of $S_k R_k^i S_k^T$ for i up to $4d + 2$. Computing $S_k R_k^i S_k^T$ from right to left (or equivalently, from left to right) as

$$S_k R_k^i S_k^T = S_k R_k (\cdots (R_k (R_k S_k^T)))$$

takes $O(n^2 p)$ time.

Using Theorem 5.1, we get the following PRISM-accelerated Newton-Schulz iteration for computing the square root,

$$\begin{aligned} X_0 &= A, Y_0 = I, R_k = I - X_k Y_k \\ X_{k+1} &= X_k g_d(R_k; \alpha_k), Y_{k+1} = g_d(R_k; \alpha_k) Y_k, \\ \text{where } \alpha_k &= \arg \min_{\alpha \in [\ell, u]} \left\| S_k \left(I - X_k^2 g_d(R_k; \alpha)^2 \right) \right\|_F^2. \end{aligned}$$

It is straightforward to see that α_k can be computed in the same way as in the sign computation, i.e., by solving the cubic equation $m'(\alpha) = 0$. The coefficients $c_0, c_1, c_2, c_3, c_4, c_5$ of the function $m(\alpha)$ have identical formulas; the only difference is that $R_k = I - X_k Y_k$ rather than the previous $R_k = I - X_k^2$. Again, $[\ell, u] = [1, 1/2]$ is recommended for $d = 1$, which corresponds to the 3rd-order Newton-Schulz iteration in Table 1; and $[\ell, u] = [3/8, 29/20]$ is recommended for $d = 2$, which corresponds to the 5th-order Newton-Schulz iteration in Table 1.

Similarly, using Theorem 5.2, we get the following PRISM-accelerated Newton-Schulz iteration for computing the polar factor UV^T , where $A = U\Sigma V^T$ is an SVD. Assume $A \in \mathbb{R}^{m \times n}$ with $m \geq n$,

$$\begin{aligned} X_0 &= A, R_k = I - X_k^T X_k, X_{k+1} = X_k g_d(R_k; \alpha_k), \\ \text{where } \alpha_k &= \arg \min_{\alpha \in [\ell, u]} \left\| S_k \left(I - X_k^2 g_d(R_k; \alpha)^2 \right) \right\|_F^2. \end{aligned}$$

Again, α_k is computed in the same way, that is, by solving the cubic equation $m'(\alpha) = 0$, where the coefficients of $m(\alpha)$ have the same formulas as in the sign computation; the only difference is that now we have $R_k = I - X_k^T X_k$ instead of $R_k = I - X_k^2$. Since Newton-Schulz for polar decomposition shares identical convergence behavior as Newton-Schulz for sign computation, the same constraint on α is recommended. That is, $[\ell, u] = [1, 1/2]$ for $d = 1$, which corresponds to the 3rd-order Newton-Schulz iteration in Table 1; and $[\ell, u] = [3/8, 29/20]$ for $d = 2$, which corresponds to the 5th-order Newton-Schulz iteration in Table 1.

A.2. DB Newton iteration for matrix square roots

We first apply Part I of the PRISM meta-algorithm to derive the Newton iteration for computing the p -th root of a matrix, and consequently we obtain the Newton iteration for matrix square root as a special case for $p = 2$. Then we apply Part II to accelerate its computation. We will assume that the input matrix A is symmetric since this is the most relevant case, for example, arising as the Hessian or the covariance matrix.

Let $a, x > 0$, and write

$$a^{1/p} = x(x^{-p}a)^{1/p} = xf(\xi),$$

where $f(\xi) = (1 - \xi)^{1/p}$ and $\xi = 1 - x^{-p}a$. This reduces the problem of approximating $a^{1/p}$ to that of approximating $f(\xi)$. Using the first-order Taylor approximation $f_1(\xi)$ around $\xi = 0$, i.e., $f_1(\xi) = 1 - \frac{1}{p}\xi$, we get an iterative procedure

$$x_{k+1} = x_k f_1(1 - x^{-p}a) = \frac{1}{p}((p-1)x_k + x_k^{1-p}a).$$

The matrix version,

$$X_{k+1} = \frac{1}{p}((p-1)X_k + X_k^{1-p}A),$$

is the Newton iteration for the p -th root (Higham et al., 2005). Part II of PRISM can be applied to accelerate Newton iteration for matrix p -th root for any $p \geq 2$, but here we focus on the special case $p = 2$, with the understanding that analogous results hold more generally for $p \geq 2$. The Newton iteration for matrix square root is thus

$$X_0 = A, X_{k+1} = \frac{1}{2}X_k + \frac{1}{2}X_k^{-1}A.$$

By applying PRISM, we can accelerate its convergence by executing the following iteration

$$\mathbf{X}_0 = \mathbf{A}, \mathbf{X}_{k+1} = (1 - \alpha_k)\mathbf{X}_k + \alpha_k\mathbf{X}_k^{-1}\mathbf{A}, \text{ where } \alpha_k = \arg \min \|\xi(\mathbf{X}_{k+1}, \mathbf{A})\|_F^2$$

and $\xi(\mathbf{X}_k, \mathbf{A}) = \mathbf{I} - \mathbf{X}_{k+1}^{-1}\mathbf{A}$ is the residual matrix at iteration $k + 1$. In practice, coupled versions of Newton iteration are often preferred to improve numerical stability. By introducing $\mathbf{Y}_k = \mathbf{A}^{-1}\mathbf{X}_k$ to the standard Newton iteration, we obtain the DB Newton iteration due to [Denman & Beavers \(1976\)](#),

$$\begin{aligned} \mathbf{X}_{k+1} &= \frac{1}{2}\mathbf{X}_k + \frac{1}{2}\mathbf{Y}_k^{-1}, \mathbf{X}_0 = \mathbf{A}, \\ \mathbf{Y}_{k+1} &= \frac{1}{2}\mathbf{Y}_k + \frac{1}{2}\mathbf{X}_k^{-1}, \mathbf{Y}_0 = \mathbf{I}. \end{aligned}$$

DB Newton requires performing two matrix inversions at each iteration. To reduce the number of matrix inversions at each iteration from two to one, we may introduce $\mathbf{M}_k = \mathbf{X}_k\mathbf{Y}_k$ and obtain the following product form of DB Newton ([Cheng et al., 2001](#)),

$$\begin{aligned} \mathbf{M}_{k+1} &= \frac{1}{2}\mathbf{I} + \frac{1}{4}\mathbf{M}_k + \frac{1}{4}\mathbf{M}_k^{-1}, \mathbf{M}_0 = \mathbf{A}, \\ \mathbf{X}_{k+1} &= \frac{1}{2}\mathbf{X}_k + \frac{1}{2}\mathbf{X}_k\mathbf{M}_k^{-1}, \mathbf{X}_0 = \mathbf{A}, \\ \mathbf{Y}_{k+1} &= \frac{1}{2}\mathbf{Y}_k + \frac{1}{2}\mathbf{Y}_k\mathbf{M}_k^{-1}, \mathbf{Y}_0 = \mathbf{I}. \end{aligned}$$

Introducing $\mathbf{M}_k = \mathbf{X}_k\mathbf{Y}_k$ to the PRISM-accelerated version of Newton iteration, we get

$$\begin{aligned} \mathbf{M}_{k+1} &= 2\alpha_k(1 - \alpha_k)\mathbf{I} + (1 - \alpha_k)^2\mathbf{M}_k + \alpha_k^2\mathbf{M}_k^{-1}, \mathbf{M}_0 = \mathbf{A}, \\ \mathbf{X}_{k+1} &= (1 - \alpha_k)\mathbf{X}_k + \alpha_k\mathbf{X}_k\mathbf{M}_k^{-1}, \mathbf{X}_0 = \mathbf{A}, \\ \mathbf{Y}_{k+1} &= (1 - \alpha_k)\mathbf{Y}_k + \alpha_k\mathbf{Y}_k\mathbf{M}_k^{-1}, \mathbf{Y}_0 = \mathbf{I}, \\ &\text{where } \alpha_k = \arg \min \|\mathbf{I} - \mathbf{M}_{k+1}\|_F^2. \end{aligned}$$

Given $\mathbf{M}_k$, the function $\|\mathbf{I} - \mathbf{M}_{k+1}\|_F^2$ is a degree-4 polynomial with respect to α_k , i.e.

$$\|\mathbf{I} - \mathbf{M}_{k+1}\|_F^2 = m(\alpha_k) = c_0 + c_1\alpha_k + c_2\alpha_k^2 + c_3\alpha_k^3 + c_4\alpha_k^4,$$

where

$$\begin{aligned} c_1 &= \text{tr}(-4\mathbf{I} + 8\mathbf{M}_k - 4\mathbf{M}_k^2), \\ c_2 &= \text{tr}(10\mathbf{I} - 14\mathbf{M}_k + 6\mathbf{M}_k^2 - 2\mathbf{M}_k^{-1}), \\ c_3 &= \text{tr}(-12\mathbf{I} + 12\mathbf{M}_k - 4\mathbf{M}_k^2 + 4\mathbf{M}_k^{-1}), \\ c_4 &= \text{tr}(6\mathbf{I} - 4\mathbf{M}_k + \mathbf{M}_k^2 - 4\mathbf{M}_k^{-1} + \mathbf{M}_k^{-2}). \end{aligned}$$

Using the linearity of matrix trace and the fact that for symmetric matrix $\mathbf{A}$,

$$\text{tr}(\mathbf{A}^2) = \sum_{i,j} \mathbf{A}_{i,j}^2,$$

all of these coefficients can be efficiently computed in $O(n^2)$ time, without having to perform matrix multiplications. Therefore, the optimal α_k that minimizes the Frobenius norm of the residual matrix for the next iterate can be efficiently computed without sketching. We note that this is a distinct difference compared with Newton-Schulz-like algorithms for computing square roots. In addition, unlike Newton-Schulz iteration for square root which has a local convergence region, since Newton iteration for matrix square root is globally convergent, we do not need to impose any interval constraint on the coefficient α_k when we solve the optimization problem.

Remark on computing the matrix inverse $\mathbf{M}_k^{-1}$. When the input matrix $\mathbf{A}$ is symmetric, one can easily verify that $\mathbf{M}_k$ is symmetric for all k . Therefore, $\mathbf{M}_k^{-1}$ can be computed via triangular solve from the Cholesky factorization of $\mathbf{M}_k$. This can greatly improve the practical runtime of the Newton iteration. In Figure 11, we show that PRISM-based Newton iteration can outperform PRISM-based Newton-Schulz by a good margin.

A.3. Inverse Newton iteration for inverse p -th root

We first apply Part I of the PRISM meta-algorithm to derive inverse Newton iteration for computing the inverse p -th root of a matrix, and then we apply Part II to accelerate its computation. We will again assume that the input matrix $\mathbf{A}$ is symmetric.

Let $a, x > 0$, and write

$$a^{-1/p} = x(x^p a)^{-1/p} = x f(\xi),$$

where $f(\xi) = (1 - \xi)^{-1/p}$ and $\xi = 1 - x^p a$. This reduces the problem of approximating $a^{-1/p}$ to that of approximating $f(\xi)$. Using the first-order Taylor approximation $f_1(\xi)$ around $\xi = 0$, i.e., $f_1(\xi) = 1 + \frac{1}{p}\xi$, we get an iterative procedure

$$x_{k+1} = x_k f_1(1 - x^p a) = \frac{1}{p}((p+1)x_k - x_k^{p+1}a).$$

The matrix version,

$$\mathbf{X} = \mathbf{A}, \mathbf{X}_{k+1} = \frac{1}{p}((p+1)\mathbf{X}_k - \mathbf{X}_k^{p+1}\mathbf{A}),$$

is the inverse Newton iteration for the inverse p -th root (Higham et al., 2005). When $p = 1$, this is a variant of the Newton-Schulz iteration for matrix inverse (Higham et al., 2005). In practice, by introducing $\mathbf{M}_k = \mathbf{X}_k^p \mathbf{A}$, the following coupled inverse Newton iteration is preferred to improve numerical stability (Higham et al., 2005),

$$\begin{aligned} \mathbf{X}_{k+1} &= \mathbf{X}_k \left(\frac{(p+1)\mathbf{I} - \mathbf{M}_k}{p} \right), \mathbf{X}_0 = \frac{1}{c}\mathbf{I}, \\ \mathbf{M}_{k+1} &= \left(\frac{(p+1)\mathbf{I} - \mathbf{M}_k}{p} \right)^p \mathbf{M}_k, \mathbf{M}_0 = \frac{1}{c^p}\mathbf{A}, \end{aligned}$$

where a good choice of c to guarantee convergence is

$$c = \left(\frac{2\|\mathbf{A}\|_F}{p+1} \right)^{1/p}.$$

By noting that the residual matrix

$$\mathbf{R}_k = \xi(\mathbf{X}_k, \mathbf{A}) = \mathbf{I} - \mathbf{X}_k^p \mathbf{A} = \mathbf{I} - \mathbf{M}_k,$$

the above coupled inverse Newton iteration can be equivalently written as

$$\begin{aligned} \mathbf{R}_k &= \mathbf{I} - \mathbf{M}_k, \mathbf{X}_{k+1} = \mathbf{X}_k \left(\mathbf{I} + \frac{1}{p}\mathbf{R}_k \right), \mathbf{X}_0 = \frac{1}{c}\mathbf{I}, \\ \mathbf{M}_{k+1} &= \left(\mathbf{I} + \frac{1}{p}\mathbf{R}_k \right)^p \mathbf{M}_k, \mathbf{M}_0 = \frac{1}{c^p}\mathbf{A}. \end{aligned}$$

Applying PRISM to this, we get

$$\begin{aligned} \mathbf{R}_k &= \mathbf{I} - \mathbf{M}_k, \mathbf{X}_{k+1} = \mathbf{X}_k(\mathbf{I} + \alpha_k \mathbf{R}_k), \mathbf{X}_0 = \frac{1}{c}\mathbf{I}, \\ \mathbf{M}_{k+1} &= (\mathbf{I} + \alpha_k \mathbf{R}_k)^p \mathbf{M}_k, \mathbf{M}_0 = \frac{1}{c^p}\mathbf{A}, \\ \text{where } \alpha_k &= \arg \min_{\alpha \in [\ell, u]} \left\| \mathbf{S}_k \left(\mathbf{R}_k + \sum_{i=1}^p \binom{p}{i} \alpha^i (\mathbf{R}_k^{i+1} - \mathbf{R}_k^i) \right) \right\|_F^2, \end{aligned}$$

and $\mathbf{S}_k \in \mathbb{R}^{m \times n}$ where $m \ll n$ is a sketch matrix, e.g., consisting of i.i.d Gaussian entries. Denote the optimization objective function in the definition of α_k as

$$m(\alpha) = \left\| \mathbf{S}_k \left(\mathbf{R}_k + \sum_{i=1}^p \binom{p}{i} \alpha^i (\mathbf{R}_k^{i+1} - \mathbf{R}_k^i) \right) \right\|_F^2.$$

The function $m(\alpha)$ is a polynomial of degree $2p$ with respect to α . For $p = 1$, we have

$$m(\alpha) = c_0 + c_1\alpha + c_2\alpha^2$$

where

$$\begin{aligned} c_1 &= 2 \operatorname{tr}(\mathbf{S}_k \mathbf{R}_k^3 \mathbf{S}_k^T) - 2 \operatorname{tr}(\mathbf{S}_k \mathbf{R}_k^2 \mathbf{S}_k^T), \\ c_2 &= \operatorname{tr}(\mathbf{S}_k \mathbf{R}_k^4 \mathbf{S}_k^T) - 2 \operatorname{tr}(\mathbf{S}_k \mathbf{R}_k^3 \mathbf{S}_k^T) + \operatorname{tr}(\mathbf{S}_k \mathbf{R}_k^2 \mathbf{S}_k^T). \end{aligned}$$

For $p = 2$, we have

$$m(\alpha) = c_0 + c_1\alpha + c_2\alpha^2 + c_3\alpha^3 + c_4\alpha^4,$$

where

$$\begin{aligned} c_1 &= 4 \operatorname{tr}(\mathbf{S}_k \mathbf{R}_k^3 \mathbf{S}_k^T) - 4 \operatorname{tr}(\mathbf{S}_k \mathbf{R}_k^2 \mathbf{S}_k^T), \\ c_2 &= 6 \operatorname{tr}(\mathbf{S}_k \mathbf{R}_k^4 \mathbf{S}_k^T) - 10 \operatorname{tr}(\mathbf{S}_k \mathbf{R}_k^3 \mathbf{S}_k^T) + 4 \operatorname{tr}(\mathbf{S}_k \mathbf{R}_k^2 \mathbf{S}_k^T), \\ c_3 &= 4 \operatorname{tr}(\mathbf{S}_k \mathbf{R}_k^5 \mathbf{S}_k^T) - 8 \operatorname{tr}(\mathbf{S}_k \mathbf{R}_k^4 \mathbf{S}_k^T) + 4 \operatorname{tr}(\mathbf{S}_k \mathbf{R}_k^3 \mathbf{S}_k^T), \\ c_4 &= \operatorname{tr}(\mathbf{S}_k \mathbf{R}_k^6 \mathbf{S}_k^T) - 2 \operatorname{tr}(\mathbf{S}_k \mathbf{R}_k^5 \mathbf{S}_k^T) + \operatorname{tr}(\mathbf{S}_k \mathbf{R}_k^4 \mathbf{S}_k^T). \end{aligned}$$

In both cases, computing the coefficients of $m(\alpha)$ takes $O(mn^2)$ time. Once the coefficients are known, α_k that minimizes $m(\alpha)$ can be computed analytically by solving $m'(\alpha) = 0$.

For $p \geq 3$, the coefficients of $m(\alpha) = \sum_{i=0}^{2p} c_i \alpha^i$ can be computed similarly in $O(mn^2)$ time, but minimizing $m(\alpha)$ requires a numerical optimization algorithm. Since this is a scalar polynomial function, this can be done by numerically solving $m'(\alpha) = 0$ and then evaluating $m(\alpha)$ at the roots of its derivative. Numerically computing the roots of $m'(\alpha)$ by computing the eigenvalues of the companion matrix takes $O(p^3)$ times.

A.4. Chebyshev's iteration for inverse

We first apply Part I of the PRISM meta-algorithm to derive Chebyshev's iteration for computing matrix inverse, and then we apply Part II to accelerate its computation. Here, we do not require the input matrix $\mathbf{A}$ to be symmetric, but we will assume that $\|\mathbf{A}\|_2 \leq 1$, which is easily satisfied by normalizing $\mathbf{A} \mapsto \mathbf{A}/\|\mathbf{A}\|_F$ for a general full-rank square matrix $\mathbf{A}$.

Let $a, x \neq 0$, and write

$$a^{-1} = x(ax)^{-1} = xf(\xi),$$

where $f(\xi) = (1 - \xi)^{-1}$ and $\xi = 1 - ax$. Using the second-order Taylor approximation $f_2(\xi)$ around $\xi = 0$, i.e., $f_2(\xi) = 1 + \xi + \xi^2$, we get an iterative procedure

$$x_{k+1} = x_k f_2(1 - ax) = 3x_k - 3x_k ax_k + x_k ax_k a_k.$$

The matrix version,

$$\mathbf{X}_0 = \mathbf{A}^T, \quad \mathbf{X}_{k+1} = \mathbf{X}_k(\mathbf{I} + \mathbf{R}_k + \mathbf{R}_k^2) = 3\mathbf{X}_k - 3\mathbf{X}_k \mathbf{A} \mathbf{X}_k + \mathbf{X}_k \mathbf{A} \mathbf{X}_k \mathbf{A} \mathbf{X}_k,$$

gives Chebyshev's iteration, where $\mathbf{R}_k = \mathbf{I} - \mathbf{A} \mathbf{X}_k$.

Applying Part II of PRISM, we get the following accelerated version,

$$\begin{aligned} \mathbf{X}_0 &= \mathbf{A}^T, \quad \mathbf{R}_k = \mathbf{I} - \mathbf{A} \mathbf{X}_k, \quad \mathbf{X}_{k+1} = \mathbf{X}_k(\mathbf{I} + \mathbf{R}_k + \alpha_k \mathbf{R}_k^2), \\ \text{where } \alpha_k &= \arg \min_{\alpha \in [\ell, u]} \left\| \mathbf{S}_k \left(\mathbf{R}_k^2 - \alpha(\mathbf{R}_k^2 - \mathbf{R}_k^3) \right) \right\|_F^2. \end{aligned}$$

Write

$$m(\alpha) = \left\| \mathbf{S}_k \left(\mathbf{R}_k^2 - \alpha(\mathbf{R}_k^2 - \mathbf{R}_k^3) \right) \right\|_F^2 = c_0 + c_1\alpha + c_2\alpha^2,$$

we have

$$\begin{aligned} c_1 &= -2 \operatorname{tr}(\mathbf{S}_k \mathbf{R}_k^4 \mathbf{S}_k^T) + 2 \operatorname{tr}(\mathbf{S}_k \mathbf{R}_k^5 \mathbf{S}_k^T) \\ c_2 &= \operatorname{tr}(\mathbf{S}_k \mathbf{R}_k^4 \mathbf{S}_k^T) - 2 \operatorname{tr}(\mathbf{S}_k \mathbf{R}_k^5 \mathbf{S}_k^T) + \operatorname{tr}(\mathbf{S}_k \mathbf{R}_k^6 \mathbf{S}_k^T). \end{aligned}$$

Therefore, in the PRISM-accelerated Chebyshev's iteration, α_k can be computed in closed-form by solving $m'(\alpha) = 0$. Empirically, we found that enforcing $\alpha_k \in [\ell, u] = [1/2, 2]$ is sufficient to ensure fast convergence.

B. Pseudocode of Newton-Schulz Variants for Polar Decomposition and Square Roots

We provide a pseudocode for the PRISM-based Newton-Schulz iterations (3rd- and 5th-order, respectively) for computing the polar factor and square roots, respectively. These follow closely with what we implemented in our experiments. Other PRISM-based algorithms, such as PRISM-based Inverse Newton for the inverse 4th root, and PRISM-based Newton iteration for square roots, can be implemented in a similar fashion. Note that, for each of these algorithms, we have already provided all necessary details in Appendix A. This section shows a bit more details (and in a more compact form, so it is easier to navigate) to make it easier for an actual implementation of PRISM-based Newton-Schulz iterations from Appendix A.1.

Notation: We use $\odot$ to denote element-wise multiplication. For a matrix $\mathbf{A}$, we use $\text{sum}(\mathbf{A})$ to denote the sum of all elements of $\mathbf{A}$.

Practical consideration on the sketch matrix. Even though our theory (e.g. Theorem 4.2) requires a different sketch matrix $\mathbf{S}_k$ at every iteration, in practice we never encounter an issue by using a single sketch matrix $\mathbf{S}$ throughout all iterations. In fact, using a single, fixed sketch matrix does not seem to harm the practical performance at all. We thus conjecture that the theoretical requirement for a different sketch matrix at every iteration may just be an artifact of our proof approach, but not really necessary. Therefore, in a practical implementation, we suggest using a fixed sketch matrix to avoid having to sample a different one at every iteration. There, in the pseudocode, we sample a fixed Gaussian sketch matrix only once at the beginning. Additionally, we fix the sketch dimension to be constant at 16. This appears to be more than enough for all practical purposes, including input matrices of dimension up to 10,000 x 10,000.

Practical consideration on computing $\text{tr}(\mathbf{S}\mathbf{A}^k\mathbf{S}^T)$. If $\mathbf{A}$ is symmetric, then for any $i, j, k \geq 1$ such that $i + j = k$, we have the identity

$$\text{tr}(\mathbf{S}\mathbf{A}^k\mathbf{S}^T) = \text{sum}((\mathbf{A}^i\mathbf{S}) \odot (\mathbf{A}^j\mathbf{S})).$$

We use this to slightly reduce the number of matrix-matrix multiplications, for example, when computing $\text{tr}(\mathbf{S}\mathbf{A}^{10}\mathbf{S}^T)$, we just need to first compute $\mathbf{A}^5\mathbf{S} = \mathbf{A}(\mathbf{A}(\mathbf{A}(\mathbf{A}(\mathbf{A}\mathbf{S}))))$, and then use $\text{tr}(\mathbf{S}\mathbf{A}^{10}\mathbf{S}^T) = \text{sum}((\mathbf{A}^5\mathbf{S}) \odot (\mathbf{A}^5\mathbf{S}))$.

B.1. Algorithm 1: PRISM Newton-Schulz (3rd order) for polar decomposition

Input: Matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, where $m \geq n$. If $m < n$, take the transpose $\mathbf{A}^T$. Constants $\ell = 0.5$ and $u = 1$. Number of iterations $K \geq 1$.

Output: The polar factor $\mathbf{O} = \mathbf{U}\mathbf{V}^T$ of $\mathbf{A}$, where $\mathbf{U} \in \mathbb{R}^{m \times n}$ and $\mathbf{V} \in \mathbb{R}^{n \times n}$ contain the left and right singular vectors of $\mathbf{A}$.

1. Sample a Gaussian sketch matrix $\mathbf{S} \in \mathbb{R}^{n \times 16}$, i.e., each entry $\mathbf{S}_{ij}$ is i.i.d. Gaussian with mean 0 and variance 1/16.
2. Initialize $\mathbf{X}_0 = \mathbf{A} / \|\mathbf{A}\|_F$.
3. For $k = 0, 1, 2, \dots, K - 1$:
 - (a) Compute the residual $\mathbf{R}_k = \mathbf{I} - \mathbf{X}_k^T \mathbf{X}_k$.
 - (b) Compute the matrix sketches $\tilde{\mathbf{R}}_k^{(1)} = \mathbf{R}_k \mathbf{S}$ and $\tilde{\mathbf{R}}_k^{(i+1)} = \mathbf{R}_k \tilde{\mathbf{R}}_k^{(i)}$ for $i = 1, 2, 3$.
 - (c) Compute the trace estimates

$$\begin{aligned} r_k^{(2)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(1)} \odot \tilde{\mathbf{R}}_k^{(1)}), & r_k^{(3)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(2)} \odot \tilde{\mathbf{R}}_k^{(1)}), & r_k^{(4)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(2)} \odot \tilde{\mathbf{R}}_k^{(2)}), \\ r_k^{(5)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(2)} \odot \tilde{\mathbf{R}}_k^{(3)}), & r_k^{(6)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(3)} \odot \tilde{\mathbf{R}}_k^{(3)}). \end{aligned}$$

- (d) Compute α_k via polynomial fitting, i.e.,

$$\alpha_k = \arg \min_{\alpha \in [\ell, u]} c_k^{(1)} \alpha + c_k^{(2)} \alpha^2 + c_k^{(3)} \alpha^3 + c_k^{(4)} \alpha^4,$$

where

$$\begin{aligned} c_k^{(1)} &= 4r_k^{(3)} - 4r_k^{(2)}, \\ c_k^{(2)} &= 6r_k^{(4)} - 10r_k^{(3)} + 4r_k^{(2)}, \\ c_k^{(3)} &= 4r_k^{(5)} - 8r_k^{(4)} + 4r_k^{(3)}, \\ c_k^{(4)} &= r_k^{(6)} - 2r_k^{(5)} + r_k^{(4)}. \end{aligned}$$

(e) Update $\mathbf{X}_{k+1} = \mathbf{X}_k(\mathbf{I} + \alpha_k \mathbf{R}_k)$.

4. Return $\mathbf{X}_K$.

B.2. Algorithm 2: PRISM Newton-Schulz (5th order) for polar decomposition

Input: Matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, where $m \geq n$. If $m < n$, take the transpose $\mathbf{A}^T$. Constants $\ell = 0.375$ and $u = 1.45$. Number of iterations $K \geq 1$.

Output: The polar factor $\mathbf{O} = \mathbf{U}\mathbf{V}^T$ of $\mathbf{A}$, where $\mathbf{U} \in \mathbb{R}^{m \times n}$ and $\mathbf{V} \in \mathbb{R}^{n \times n}$ contain the left and right singular vectors of $\mathbf{A}$.

1. Sample a Gaussian sketch matrix $\mathbf{S} \in \mathbb{R}^{n \times 16}$, i.e., each entry $\mathbf{S}_{ij}$ is i.i.d. Gaussian with mean 0 and variance 1/16.

2. Initialize $\mathbf{X}_0 = \mathbf{A}/\|\mathbf{A}\|_F$.

3. For $k = 0, 1, 2, \dots, K - 1$:

(a) Compute the residual $\mathbf{R}_k = \mathbf{I} - \mathbf{X}_k^T \mathbf{X}_k$.

(b) Compute the matrix sketches $\tilde{\mathbf{R}}_k^{(1)} = \mathbf{R}_k \mathbf{S}$ and $\tilde{\mathbf{R}}_k^{(i+1)} = \mathbf{R}_k \tilde{\mathbf{R}}_k^{(i)}$ for $i = 1, 2, 3, 4$.

(c) Compute the trace estimates

$$\begin{aligned} r_k^{(2)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(1)} \odot \tilde{\mathbf{R}}_k^{(1)}), & r_k^{(3)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(2)} \odot \tilde{\mathbf{R}}_k^{(1)}), & r_k^{(4)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(2)} \odot \tilde{\mathbf{R}}_k^{(2)}), \\ r_k^{(5)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(2)} \odot \tilde{\mathbf{R}}_k^{(3)}), & r_k^{(6)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(3)} \odot \tilde{\mathbf{R}}_k^{(3)}), & r_k^{(7)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(3)} \odot \tilde{\mathbf{R}}_k^{(4)}), \\ r_k^{(8)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(4)} \odot \tilde{\mathbf{R}}_k^{(4)}), & r_k^{(9)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(4)} \odot \tilde{\mathbf{R}}_k^{(5)}), & r_k^{(10)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(5)} \odot \tilde{\mathbf{R}}_k^{(5)}). \end{aligned}$$

(d) Compute α_k via polynomial fitting, i.e.,

$$\alpha_k = \arg \min_{\alpha \in [\ell, u]} c_k^{(1)} \alpha + c_k^{(2)} \alpha^2 + c_k^{(3)} \alpha^3 + c_k^{(4)} \alpha^4,$$

where

$$\begin{aligned} c_k^{(1)} &= 0.5r_k^{(7)} + 2r_k^{(6)} + 0.5r_k^{(5)} - 3 * r_k^{(4)}, \\ c_k^{(2)} &= 1.5r_k^{(8)} + 3r_k^{(7)} - 4.5r_k^{(6)} - 4r_k^{(5)} + 4r_k^{(4)}, \\ c_k^{(3)} &= 2r_k^{(9)} - 6r_k^{(7)} + 4r_k^{(6)}, \\ c_k^{(4)} &= r_k^{(10)} - 2r_k^{(9)} + r_k^{(8)}. \end{aligned}$$

(e) Update $\mathbf{X}_{k+1} = \mathbf{X}_k(\mathbf{I} + 0.5\mathbf{R}_k + \alpha_k \mathbf{R}_k^2)$.

4. Return $\mathbf{X}_K$.

B.3. Algorithm 3: PRISM Newton-Schulz (3rd order) for square roots

Input: Symmetric matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$. Constants $\ell = 0.5$ and $u = 1$. Number of iterations $K \geq 1$.

Output: $\mathbf{A}^{1/2}$ and $\mathbf{A}^{-1/2}$.

1. Sample a Gaussian sketch matrix $\mathbf{S} \in \mathbb{R}^{n \times 16}$, i.e., each entry $\mathbf{S}_{ij}$ is i.i.d. Gaussian with mean 0 and variance 1/16.

2. Let $c = \|\mathbf{A}\|_F$. Initialize $\mathbf{X}_0 = \mathbf{A}/c$, $\mathbf{Y}_0 = \mathbf{I}$.

3. For $k = 0, 1, 2, \dots, K - 1$:

(a) Compute the residual $\mathbf{R}_k = \mathbf{I} - \mathbf{Y}_k^T \mathbf{X}_k$.

(b) Compute the matrix sketches $\tilde{\mathbf{R}}_k^{(1)} = \mathbf{R}_k \mathbf{S}$ and $\tilde{\mathbf{R}}_k^{(i+1)} = \mathbf{R}_k \tilde{\mathbf{R}}_k^{(i)}$ for $i = 1, 2, 3$.

(c) Compute the trace estimates

$$\begin{aligned} r_k^{(2)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(1)} \odot \tilde{\mathbf{R}}_k^{(1)}), & r_k^{(3)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(2)} \odot \tilde{\mathbf{R}}_k^{(1)}), & r_k^{(4)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(2)} \odot \tilde{\mathbf{R}}_k^{(2)}), \\ r_k^{(5)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(2)} \odot \tilde{\mathbf{R}}_k^{(3)}), & r_k^{(6)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(3)} \odot \tilde{\mathbf{R}}_k^{(3)}). \end{aligned}$$

(d) Compute α_k via polynomial fitting, i.e.,

$$\alpha_k = \arg \min_{\alpha \in [\ell, u]} c_k^{(1)} \alpha + c_k^{(2)} \alpha^2 + c_k^{(3)} \alpha^3 + c_k^{(4)} \alpha^4,$$

where

$$\begin{aligned} c_k^{(1)} &= 4r_k^{(3)} - 4r_k^{(2)}, \\ c_k^{(2)} &= 6r_k^{(4)} - 10r_k^{(3)} + 4r_k^{(2)}, \\ c_k^{(3)} &= 4r_k^{(5)} - 8r_k^{(4)} + 4r_k^{(3)}, \\ c_k^{(4)} &= r_k^{(6)} - 2r_k^{(5)} + r_k^{(4)}. \end{aligned}$$

(e) Update

$$\mathbf{Z}_k = \mathbf{I} + \alpha_k \mathbf{R}_k, \quad \mathbf{X}_{k+1} = \mathbf{X}_k \mathbf{Z}_k, \quad \mathbf{Y}_{k+1} = \mathbf{Z}_k \mathbf{Y}_k.$$

4. Return $\sqrt{c} \mathbf{X}_K \approx \mathbf{A}^{1/2}$, $\sqrt{c}^{-1} \mathbf{Y}_K \approx \mathbf{A}^{-1/2}$.

B.4. Algorithm 4: PRISM Newton-Schulz (5th order) for square roots

Input: Symmetric matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$. Constants $\ell = 0.5$ and $u = 1$. Number of iterations $K \geq 1$.

Output: $\mathbf{A}^{1/2}$ and $\mathbf{A}^{-1/2}$.

1. Sample a Gaussian sketch matrix $\mathbf{S} \in \mathbb{R}^{n \times 16}$, i.e., each entry $\mathbf{S}_{ij}$ is i.i.d. Gaussian with mean 0 and variance 1/16.
2. Let $c = \|\mathbf{A}\|_F$. Initialize $\mathbf{X}_0 = \mathbf{A}/c$, $\mathbf{Y}_0 = \mathbf{I}$.
3. For $k = 0, 1, 2, \dots, K-1$:

(a) Compute the residual $\mathbf{R}_k = \mathbf{I} - \mathbf{Y}_k^T \mathbf{X}_k$.

(b) Compute the matrix sketches $\tilde{\mathbf{R}}_k^{(1)} = \mathbf{R}_k \mathbf{S}$ and $\tilde{\mathbf{R}}_k^{(i+1)} = \mathbf{R}_k \tilde{\mathbf{R}}_k^{(i)}$ for $i = 1, 2, 3, 4$.

(c) Compute the trace estimates

$$\begin{aligned} r_k^{(2)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(1)} \odot \tilde{\mathbf{R}}_k^{(1)}), & r_k^{(3)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(2)} \odot \tilde{\mathbf{R}}_k^{(1)}), & r_k^{(4)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(2)} \odot \tilde{\mathbf{R}}_k^{(2)}), \\ r_k^{(5)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(2)} \odot \tilde{\mathbf{R}}_k^{(3)}), & r_k^{(6)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(3)} \odot \tilde{\mathbf{R}}_k^{(3)}), & r_k^{(7)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(3)} \odot \tilde{\mathbf{R}}_k^{(4)}), \\ r_k^{(8)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(4)} \odot \tilde{\mathbf{R}}_k^{(4)}), & r_k^{(9)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(4)} \odot \tilde{\mathbf{R}}_k^{(5)}), & r_k^{(10)} &= \text{sum}(\tilde{\mathbf{R}}_k^{(5)} \odot \tilde{\mathbf{R}}_k^{(5)}). \end{aligned}$$

(d) Compute α_k via polynomial fitting, i.e.,

$$\alpha_k = \arg \min_{\alpha \in [\ell, u]} c_k^{(1)} \alpha + c_k^{(2)} \alpha^2 + c_k^{(3)} \alpha^3 + c_k^{(4)} \alpha^4,$$

where

$$\begin{aligned} c_k^{(1)} &= 0.5r_k^{(7)} + 2r_k^{(6)} + 0.5r_k^{(5)} - 3 * r_k^{(4)}, \\ c_k^{(2)} &= 1.5r_k^{(8)} + 3r_k^{(7)} - 4.5r_k^{(6)} - 4r_k^{(5)} + 4r_k^{(4)}, \\ c_k^{(3)} &= 2r_k^{(9)} - 6r_k^{(7)} + 4r_k^{(6)}, \\ c_k^{(4)} &= r_k^{(10)} - 2r_k^{(9)} + r_k^{(8)}. \end{aligned}$$

(e) Update

$$\mathbf{Z}_k = \mathbf{I} + 0.5\mathbf{R}_k + \alpha_k \mathbf{R}_k^2, \quad \mathbf{X}_{k+1} = \mathbf{X}_k \mathbf{Z}_k, \quad \mathbf{Y}_{k+1} = \mathbf{Z}_k \mathbf{Y}_k.$$

4. Return $\sqrt{c} \mathbf{X}_K \approx \mathbf{A}^{1/2}$, $\sqrt{c}^{-1} \mathbf{Y}_K \approx \mathbf{A}^{-1/2}$.

C. Proof of Theorems in the Main Text

C.1. Technical lemma

The proofs rely heavily on the following lemma which summarizes some important properties of the polynomial

$$h(x, \alpha) = 1 - (1 - x)(1 + \alpha x)^2.$$

Lemma C.1. *The function $h(x, \alpha) = 1 - (1 - x)(1 + \alpha x)^2$ has the following properties:*

1. $h(x, \alpha) \in [-1/5, x^2]$ for all $x \in [1/2, 1]$ and for all $\alpha \in [1/2, 1]$;
2. $h(x, \alpha) \in [-1/5, 1/4]$ for all $x \in [-1/5, 1/2]$ and for all $\alpha \in [1/2, 1]$;
3. Let $n \geq 1$ and $x_1, x_2, \dots, x_n \in [-1/4, 1/4]$ and $\alpha^* = \arg \min_{\alpha \in [1/2, 1]} \sum_{i=1}^n h(x_i, \alpha)^2$, we have

$$\max_i |h(x_i, \alpha^*)| \leq C \max_i x_i^2 \text{ for some constant } C < 1.71.$$

4. Let $n \geq 1$ and $x_1, x_2, \dots, x_n \in [-1/4, 1/4]$, not all 0, and $\alpha^* = \arg \min_{\alpha \in [1/2, 1]} \sum_{i=1}^n h(x_i, \alpha)^2$ and $\tilde{\alpha} \in [1/2, 1]$ be such that $\sum_{i=1}^n h(x_i, \tilde{\alpha})^2 \leq (1 + \gamma) \sum_{i=1}^n h(x_i, \alpha^*)^2$ for some $\gamma \geq 0$. Then

$$|\alpha^* - \tilde{\alpha}| < \sqrt{\gamma} D \max_i |x_i| \text{ for some constant } D < 0.51.$$

5. Let $n \geq 1$ and $x_1, x_2, \dots, x_n \in [-1/4, 1/4]$, and let $\tilde{\alpha} \in [1/2, 1]$ be such that $\sum_{i=1}^n h(x_i, \tilde{\alpha})^2 \leq (1 + \gamma) \sum_{i=1}^n h(x_i, \alpha)^2$ for all $\alpha \in [1/2, 1]$, where $\gamma < 1.38$. Then

$$\max_i |h(x_i, \tilde{\alpha})| \leq E \max_i x_i^2 \text{ for some constant } E < 2.95.$$

We break the proof of Lemma C.1 into separate claims and prove each claim separately.

Claim 1. $h(x, \alpha) \in [-1/5, x^2]$ for all $x \in [1/2, 1]$ and for all $\alpha \in [1/2, 1]$.

Proof. Differentiate $h(x, \alpha)$ with respect to α , we get that for all $x \in [1/2, 1]$ and for all $\alpha \in [1/2, 1]$,

$$\frac{\partial h}{\partial \alpha}(x, \alpha) = -2x(1 - x)(1 + \alpha x) \leq 0.$$

Therefore, the function $\alpha \mapsto h(x, \alpha)$ is monotonically decreasing on the interval $\alpha \in [1/2, 1]$. In particular,

$$h(x, 1) \leq h(x, \alpha) \leq h(x, 1/2), \quad \forall \alpha \in [1/2, 1], \quad \forall x \in [1/2, 1].$$

Thus, it suffices to show that

$$h(x, 1/2) \leq x^2 \text{ and } h(x, 1) \geq -1/5, \quad \forall x \in [1/2, 1].$$

It is easy to verify that for all $x \in [1/2, 1]$,

$$h(x, 1/2) = \frac{3}{4}x^2 + \frac{1}{4}x^3 \leq x^2.$$

To see that for all $x \in [1/2, 1]$ one also has

$$h(x, 1) = -x + x^2 + x^3 \geq -1/5,$$

we note that the derivative with respect to x ,

$$\frac{\partial h}{\partial x}(x, 1) = -1 + 2x + 3x^2 \geq 0, \quad \forall x \in [1/2, 1],$$

so $h(x, 1)$ is increasing on the interval $[1/2, 1]$, and thus for all $x \in [1/2, 1]$ one has

$$h(x, 1) \geq h(1/2, 1) = -\frac{1}{2} + \frac{1}{4} + \frac{1}{8} = -\frac{1}{8} \geq -\frac{1}{5}.$$

This completes the proof. □

Claim 2. $h(x, \alpha) \in [-1/5, 1/4]$ for all $x \in [-1/5, 1/2]$ and for all $\alpha \in [1/2, 1]$.

Proof. Define

$$g(x, \alpha) = (1 - x)(1 + \alpha x)^2,$$

so that $h(x, \alpha) = 1 - g(x, \alpha)$. The required result is equivalent to

$$\frac{3}{4} \leq g(x, \alpha) \leq \frac{6}{5},$$

which we will show in the next. Differentiate $g(x, \alpha)$ with respect to α ,

$$\frac{\partial g}{\partial \alpha}(x, \alpha) = 2x(1 - x)(1 + \alpha x).$$

For $x \in [-1/5, 1/2]$ and $\alpha \in [1/2, 1]$ we have $1 - x \geq 1/2 > 0$ and $1 + \alpha x \geq 4/5 > 0$. Hence the sign of $\partial g / \partial \alpha$ is the sign of x . Consequently,

- if $x \in [0, 1/2]$, then $g(x, \alpha)$ is increasing in α ;
- if $x \in [-1/5, 0]$, then $g(x, \alpha)$ is decreasing in α .

Therefore, for fixed x , the extrema of $g(x, \alpha)$ on $\alpha \in [1/2, 1]$ are attained at $\alpha = 1/2$ or $\alpha = 1$. Define

$$\begin{aligned} \phi(x) &= g(x, 1/2) = 1 - \frac{3}{4}x^2 - \frac{1}{4}x^3, \\ \psi(x) &= g(x, 1) = 1 + x - x^2 - x^3. \end{aligned}$$

We consider two cases depending on if $x \in [0, 1/2]$ or $x \in [-1/5, 0]$. If $x \in [0, 1/2]$, then the minimum occurs at $\alpha = 1/2$ and the maximum occurs at $\alpha = 1$. That is,

$$\phi(x) \leq g(x, \alpha) \leq \psi(x).$$

We have

$$\phi'(x) = -3x(2 + x)/4 \leq 0, \quad \forall x \in [0, 1/2],$$

so $\phi(x)$ is decreasing on $[0, 1/2]$. Hence

$$g(x, \alpha) \geq \phi(x) \geq \phi(1/2) = \frac{25}{32} > \frac{3}{4}, \quad \forall x \in [0, 1/2], \quad \forall \alpha \in [1/2, 1].$$

Furthermore,

$$\psi'(x) = 1 - 2x - 3x^2 = (1 - 3x)(1 + x)$$

has a root at $x = 1/3$ in the interval $[0, 1/2]$. This gives

$$g(x, \alpha) \leq \psi(x) \leq \max\{\psi(0), \psi(1/3), \psi(1/2)\} = \frac{32}{27} < \frac{6}{5}, \quad \forall x \in [0, 1/2], \quad \forall \alpha \in [1/2, 1].$$

On the other hand, if $x \in [-1/5, 0]$, then the minimum occurs at $\alpha = 1$ and the maximum occurs at $\alpha = 1/2$. That is,

$$\psi(x) \leq g(x, \alpha) \leq \phi(x).$$

Over the interval $[-1/5, 0]$, the derivative $\psi'(x) = (1 - 3x)(1 + x) \geq 0$, so $\psi(x)$ is increasing, and hence

$$g(x, \alpha) \geq \psi(x) \geq \psi(-1/5) = \frac{96}{125} > \frac{3}{4}, \quad \forall x \in [-1/5, 0], \quad \forall \alpha \in [1/2, 1].$$

Similarly, the derivative $\phi'(x) = -3x(2 + x)/4 \leq 0$, so $\phi(x)$ is decreasing over the interval $[-1/5, 0]$, and hence

$$g(x, \alpha) \leq \phi(x) \leq \phi(0) = 1 < \frac{6}{5}, \quad \forall x \in [-1/5, 0], \quad \forall \alpha \in [1/2, 1].$$

We obtain the required result by combining both cases. □

Claim 3. Let $n \geq 1$ and $x_1, x_2, \dots, x_n \in [-1/4, 1/4]$ and $\alpha^* = \arg \min_{\alpha \in [1/2, 1]} \sum_{i=1}^n h(x_i, \alpha)^2$, we have

$$\max_i |h(x_i, \alpha^*)| \leq C \max_i x_i^2 \text{ for some constant } C < 1.71.$$

Proof. We assume that $\max_i x_i^2 > 0$, as otherwise $x_i = h(x_i, \alpha) = 0$ for all i and for all α , and therefore the result holds trivially. To prove the result when not all x_i 's are 0, we will construct a worst-case configuration in which the ratio $\max_i |h(x_i, \alpha^*)| / \max_i x_i^2$ is maximized as much as possible, and then we will bound the maximum ratio based on the configuration. The proof uses various monotone behaviors of $h(x, \alpha)$ with respect to x or α and reducing the analysis to characterizing the worst-case ratio around an “outlier”, or “anchor point”, $x_o = -1/4$.

We start by characterizing the regions on which $|h(x, \alpha)|$ is increasing or decreasing. Fix α in $[1/2, 1]$ and consider

$$g_\alpha(x) = h(x, \alpha).$$

Then we have that

$$g'_\alpha(x) = 1 - 2\alpha + 4\alpha x - 2\alpha^2 x + 3\alpha^2 x^2 = (1 + \alpha x)(1 - 2\alpha + 3\alpha x),$$

and

$$\frac{d}{dx} |h(x, \alpha)| = \text{sign}(g_\alpha(x)) \cdot g'_\alpha(x).$$

By factoring

$$g_\alpha(x) = x(\alpha^2 x^2 + (2\alpha - \alpha^2)x + (1 - 2\alpha))$$

we see that the three roots of $g_\alpha(x)$ are

$$r_-(\alpha) = \frac{\alpha - 2 - \sqrt{\alpha(\alpha + 4)}}{2\alpha}, \quad r_0(\alpha) = 0, \quad r_+(\alpha) = \frac{\alpha - 2 + \sqrt{\alpha(\alpha + 4)}}{2\alpha}.$$

Because $\alpha \in [1/2, 1]$ we get

$$r_-(\alpha) < 0 < r_+(\alpha).$$

Since $g_\alpha(x)$ is a cubic function with positive leading coefficient,

$$\text{sign}(g_\alpha(x)) = \begin{cases} 1, & \text{if } x \in (r_-(\alpha), 0) \cup (r_+(\alpha), +\infty), \\ -1, & \text{if } x \in (-\infty, r_-(\alpha)) \cup (0, r_+(\alpha)). \end{cases}$$

On the other hand, the two roots of $g'_\alpha(x)$ are

$$x_-(\alpha) = -\frac{1}{\alpha}, \quad x_+(\alpha) = \frac{2\alpha - 1}{3\alpha},$$

and since $\alpha \in [1/2, 1]$ we get that

$$\begin{aligned} g'_\alpha(x) &> 0 \text{ for } x \in (-\infty, x_-(\alpha)) \cup (x_+(\alpha), +\infty), \\ g'_\alpha(x) &< 0 \text{ for } x \in (x_-(\alpha), x_+(\alpha)). \end{aligned}$$

Combining these, we get for a fixed $\alpha \in [1/2, 1]$,

- $|h(x, \alpha)|$ is decreasing with respect to x on $(-\infty, r_-(\alpha)) \cup (x_-(\alpha), 0) \cup (x_+(\alpha), r_+(\alpha))$;
- $|h(x, \alpha)|$ is increasing with respect to x on $(r_-(\alpha), x_-(\alpha)) \cup (0, x_+(\alpha)) \cup (r_+(\alpha), +\infty)$.

For $x_1, x_2, \dots, x_n \in [-1/4, 1/4]$, denote

$$M = \max_i |x_i| \leq 1/4,$$

so we have

$$x_i \in [-M, M] \text{ for all } i.$$

Since $r_-(\alpha) \leq x_-(\alpha) \leq -1 < -M$ for every $\alpha \in [1/2, 1]$, the monotone behavior of $|h(x, \alpha)|$ implies that

$$\begin{aligned} \frac{\max_i |h(x_i, \alpha^*)|}{\max_i x_i^2} &= \frac{\max_i |h(x_i, \alpha^*)|}{M^2} \\ &\leq \max \left\{ \frac{|h(-M, \alpha^*)|}{M^2}, \frac{|h(\min\{x_+(\alpha^*), M\}, \alpha^*)|}{M^2}, \frac{|h(M, \alpha^*)|}{M^2} \right\}. \end{aligned}$$

To bound the above quantity, for each choice of a potential “outlier”

$$x_o \in \{-M, \min\{x_+(\alpha^*), M\}, M\},$$

we find an upper bound on the maximum possible value of $|h(x_o, \alpha^*)|$ as a function of M , and then maximize the ratio $|h(x_o, \alpha^*)|/M^2$ over $M \in (0, 1/4]$. It turns out that, the case

$$x_o = -M.$$

gives rise to the worst-case ratio. We will focus on this case for the rest of the proof. The other cases all follow from a similar line of reasoning.

Since $-1/4 \leq x_o < 0$, it is straightforward to check that

$$\frac{d}{d\alpha} h(x_o, \alpha)^2 = -4x_o(1 - x_o)(1 + \alpha x_o)h(x_o, \alpha) \geq 0$$

for all $\alpha \in [1/2, 1]$. This means that the function $|h(x_o, \alpha)|$ is increasing with respect to α for $\alpha \in [1/2, 1]$, and hence $|h(x_o, \alpha^*)|$ is maximized if α^* is away from $1/2$ as far as possible. Recall that

$$\alpha^* = \arg \min_{\alpha \in [1/2, 1]} \sum_{i=1}^n h(x_i, \alpha)^2.$$

Using the definition of α^* , we now determine how large α^* can be, and consequently we derive an upper bound on $|h(x_o, \alpha^*)|$. Fix an arbitrary $x \in [-M, M]$ and consider

$$f_x(\alpha) = h(x, \alpha)^2.$$

We will show that $f_x(\alpha)$ is increasing on the interval $[\beta(x), 1]$ where

$$\beta(x) = \frac{1/\sqrt{1-x} - 1}{x},$$

and hence conclude that $\alpha^* \leq \beta(M)$. Note that if $x = 0$ then $f_x(\alpha) = 0$ for all α . So we consider two cases depending on the sign of x . If $x \in (0, 1/4]$, then by analyzing the sign of

$$f'_x(\alpha) = -4x(1-x)(1+\alpha x)h(x, \alpha)$$

we get that $\text{sign}(f'_x(\alpha)) = -\text{sign}(h(x, \alpha))$ for $\alpha \in [1/2, 1]$. It then follows from a straightforward analysis of $\text{sign}(h(x, \alpha))$ that

$$\text{sign}(f'_x(\alpha)) = \begin{cases} -1, & \text{if } \alpha \in [1/2, \beta(x)], \\ 1, & \text{if } \alpha \in [\beta(x), 1]. \end{cases}$$

Similarly, if $x \in [-1/4, 0)$, then for all $\alpha \in [1/2, 1]$ we have

$$\text{sign}(f'_x(\alpha)) = \text{sign}(h(x, \alpha)) = 1.$$

Combining both cases, we get that $f_x(\alpha)$ is monotonically increasing with respect to α for all $x \in [-1/4, 1/4]$ and for all $\alpha \in [\beta(x), 1]$. Because $\beta(x) \leq \beta(M)$ for all $x \in [-M, M]$, we must have

$$\alpha^* \leq \beta(M),$$

as otherwise one may take $\alpha = \beta(M) < \alpha^*$ to get $\sum_{i=1}^n h(x_i, \alpha) < \sum_{i=1}^n h(x_i, \alpha^*)$, contradicting the definition of α^* . Therefore, by combining the monotone increasing property of $|h(x_o, \alpha)|$ over the interval $\alpha \in [1/2, 1]$, we get

$$|h(x_o, \alpha^*)| \leq |h(x_o, \beta(M))|.$$

Since $x_o = -M$, all it left is to compute

$$\max_{M \in (0, 1/4]} \frac{|h(-M, \beta(M))|}{M^2}, \text{ where } \beta(M) = \frac{1/\sqrt{1-M} - 1}{M}.$$

Because $|h(-M, \beta(M))| \geq 0$ for all $M \in (0, 1/4]$, we will equivalently consider $h(-M, \beta(M))$ in place of $|h(-M, \beta(M))|$. Define

$$R(M) = \frac{h(-M, \beta(M))}{M^2}.$$

Consider the change of variable

$$u = \frac{1}{\sqrt{1-M}}$$

which maps $M \in (0, 1/4]$ to $u \in (1, 2/\sqrt{3}]$, and

$$h(-M, \beta(M)) = 1 - (1+M)(1 - \beta(M)M) = 1 - (1+M)(2-u)^2.$$

Dividing by M^2 and then using $M = 1 - 1/u^2$, we get

$$R(M) = r(u) = -\frac{2u^2(u^2 - 2u - 2)}{(u+1)^2}.$$

And since

$$r'(u) = -\frac{4u(u+2)(u^2 - u - 1)}{(u+1)^3} > 0$$

for all $u \in [1, 2/\sqrt{3}]$, the function $r(u)$ is strictly increasing. Therefore, the maximum of $R(M)$ is attained at

$$M = 1 - \frac{1}{(2/\sqrt{3})^2} = \frac{1}{4}.$$

Finally, for $M = 1/4$, we have

$$\beta(M) = \beta(1/4) = \frac{1/\sqrt{1-1/4} - 1}{1/4} = \frac{8}{\sqrt{3}} - 4 \in [1/2, 1],$$

and

$$R(M) = R(1/4) = \frac{h(-1/4, \beta(1/4))}{1/16} = 16 \left(-\frac{17}{3} + \frac{10}{\sqrt{3}} \right) < 1.71$$

This is an upper bound of $R(M)$ for all $M \in (0, 1/4]$, and hence the proof is complete. $\square$

Claim 4. Let $n \geq 1$ and $x_1, x_2, \dots, x_n \in [-1/4, 1/4]$, not all 0, and $\alpha^* = \arg \min_{\alpha \in [1/2, 1]} \sum_{i=1}^n h(x_i, \alpha)^2$ and $\tilde{\alpha} \in [1/2, 1]$ is such that $\sum_{i=1}^n h(x_i, \tilde{\alpha})^2 \leq (1+\gamma) \sum_{i=1}^n h(x_i, \alpha^*)^2$ for some $\gamma \geq 0$. Then

$$|\alpha^* - \tilde{\alpha}| < \sqrt{\gamma} D \max_i |x_i| \text{ for some constant } D < 0.51.$$

Proof. Denote

$$L(\alpha) = \sum_{i=1}^n h(x_i, \alpha)^2 \quad \text{and} \quad S = \sum_{i=1}^n x_i^2.$$

Since not all x_i 's are 0, we know that $S > 0$. To bound the distance between α^* and $\tilde{\alpha}$, we will use the strong convexity of the function $L(\alpha)$. We start by computing a strong convexity parameter of $L(\alpha)$ by lower bounding its second-order derivative

$L''(\alpha)$. The computation is elementary and relies on characterizing the behaviors of a couple of related polynomials in x and α .

For fixed x , define

$$f_x(\alpha) = h(x, \alpha).$$

We have

$$f'_x(\alpha) = -2x(1-x)(1+\alpha x), \quad f''_x(\alpha) = -2x^2(1-x).$$

Therefore,

$$\frac{d^2}{d\alpha^2}(f_x(\alpha)^2) = 2f'_x(\alpha)^2 + 2f_x(\alpha)f''_x(\alpha) = 4x^2(1-x)\left(3(1-x)(1+\alpha x)^2 - 1\right).$$

Let us write this as

$$\frac{d^2}{d\alpha^2}(f_x(\alpha)^2) = x^2 g(x, \alpha), \text{ where } g(x, \alpha) = 4(1-x)\left(3(1-x)(1+\alpha x)^2 - 1\right),$$

and hence

$$L''(\alpha) = \sum_{i=1}^n x_i^2 g(x_i, \alpha).$$

We will show that

$$g(x, \alpha) \geq g(1/4, 1/2) = \frac{1419}{256}, \quad \forall x \in [-1/4, 1/4], \quad \forall \alpha \in [1/2, 1].$$

Rewrite $g(x, \alpha)$ as

$$g(x, \alpha) = 12(1-x)^2(1+\alpha x)^2 - 4(1-x).$$

Differentiate $g(x, \alpha)$ with respect to α ,

$$\frac{\partial g}{\partial \alpha}(x, \alpha) = 24(1-x)^2 x (1+\alpha x).$$

For $x \in [-1/4, 1/4]$ and $\alpha \in [1/2, 1]$ we have $1-x \geq 3/4 > 0$ and $1+\alpha x \geq 3/4 > 0$, so the sign of $\partial g/\partial \alpha$ is the sign of x :

- if $x > 0$, then $\partial g/\partial \alpha > 0$ and $g(x, \alpha)$ is increasing in α , hence it attains minimum at $\alpha = 1/2$;
- if $x < 0$, then $\partial g/\partial \alpha < 0$ and $g(x, \alpha)$ is decreasing in α , hence it attains minimum at $\alpha = 1$.

This means that the minimum value of $g(x, \alpha)$ for $x \in [-1/4, 1/4]$ and $\alpha \in [1/2, 1]$ is attained on one of the following two sets

$$\{(x, 1/2) : x \in [0, 1/4]\} \text{ and } \{(x, 1) : x \in [-1/4, 0]\}.$$

We examine each case separately. Consider the case $\alpha = 1/2$ and $x \in [0, 1/4]$. Define

$$g_1(x) = g(x, 1/2) = 12(1-x)^2(1+x/2)^2 - 4(1-x) = 3x^4 + 6x^3 - 9x^2 - 8x + 8.$$

We have that

$$g'_1(x) = 12x^3 + 18x^2 - 18x - 8 \text{ and } g''_1(x) = 36x^2 + 36x - 18.$$

The function $g''_1(x)$ has two roots $x_- = (-1 - \sqrt{3})/2$ and $x_+ = (-1 + \sqrt{3})/2$. Since $x_- < 0$ and $x_+ > 1/4$ and g''_1 is a convex quadratic function, we know that $g''_1(x) < 0$ for all $x \in [0, 1/4]$, which implies that g'_1 is strictly decreasing on $[0, 1/4]$. Therefore, the maximum of $g'_1(x)$ on $[0, 1/4]$ occurs at $x = 0$. Since

$$g'_1(0) = -8 < 0$$

we get $g'_1(x) < 0$ on $[0, 1/4]$, so g_1 is strictly decreasing, and hence

$$\min_{x \in [0, 1/4]} g(x, 1/2) = \min_{x \in [0, 1/4]} g_1(x) = g_1(1/4) = \frac{1419}{256}.$$

On the other hand, consider the case $\alpha = 1$ and $x \in [-1/4, 0]$. Define

$$g_2(x) = g(x, 1) = 12(1-x)^2(1+x)^2 - 4(1-x) = 12x^4 - 24x^2 + 4x + 8.$$

Then

$$g_2'(x) = 48x^3 - 48x + 4 \text{ and } g_2''(x) = 144x^2 - 48.$$

Since $g_2''(x) = 48(3x^2 - 1) < 0$ for $x \in [-1/4, 0]$, we know that $g_2'(x)$ is strictly decreasing, so

$$g_2'(x) \geq g_2'(0) = 4 > 0 \text{ for } x \in [-1/4, 0].$$

This means that $g_2(x)$ is strictly increasing on $[-1/4, 0]$, and consequently

$$\min_{x \in [-1/4, 0]} g(x, 1) = \min_{x \in [-1/4, 0]} g_2(x) = g_2(-1/4) = \frac{355}{64}.$$

Combining both cases we get that

$$g(x, \alpha) \geq \frac{1419}{256} \text{ for } x \in [-1/4, 1/4] \text{ and } \alpha \in [1/2, 1].$$

Consequently, we get

$$L''(\alpha) \geq \frac{1419}{256} \sum_{i=1}^n x_i^2 = \frac{1419}{256} S.$$

Hence L is μ -strongly convex on $[1/2, 1]$ with

$$\mu = \frac{1419}{256} S.$$

The strong convexity of $L(\alpha)$ implies that for all $\alpha \in [1/2, 1]$,

$$L(\alpha) \geq L(\alpha^*) + \frac{\mu}{2}(\alpha - \alpha^*)^2.$$

Applying this at $\alpha = \tilde{\alpha}$ gives

$$\frac{\mu}{2}(\tilde{\alpha} - \alpha^*)^2 \leq L(\tilde{\alpha}) - L(\alpha^*).$$

Since $L(\tilde{\alpha}) \leq (1 + \gamma)L(\alpha^*)$,

$$L(\tilde{\alpha}) - L(\alpha^*) \leq \gamma L(\alpha^*),$$

and therefore

$$|\tilde{\alpha} - \alpha^*| \leq \sqrt{\frac{2\gamma L(\alpha^*)}{\mu}} = \sqrt{\frac{512}{1419}} \sqrt{\frac{\gamma L(\alpha^*)}{S}}.$$

Now, since

$$h(x, 1/2) = x^2 \left(\frac{3}{4} + \frac{x}{4} \right),$$

so

$$h(x, 1/2)^2 = x^4 \left(\frac{3}{4} + \frac{x}{4} \right)^2 \leq \frac{169}{256} x^4 \text{ for } x \in [-1/4, 1/4].$$

Therefore, by invoking the definition of α^* and M and S we get

$$L(\alpha^*) \leq L(1/2) = \sum_{i=1}^n h(x_i, 1/2)^2 \leq \frac{169}{256} \sum_{i=1}^n x_i^4 \leq \frac{169}{256} M^2 \sum_{i=1}^n x_i^2 = \frac{169}{256} M^2 S.$$

It then follows that

$$|\tilde{\alpha} - \alpha^*| \leq \sqrt{\frac{512}{1419}} \sqrt{\frac{\gamma L(\alpha^*)}{S}} \leq \sqrt{\frac{338}{1419}} \sqrt{\gamma} M < 0.51 \sqrt{\gamma} M.$$

This proves the claim. $\square$

Claim 5. Let $n \geq 1$ and $x_1, x_2, \dots, x_n \in [-1/4, 1/4]$, and let $\tilde{\alpha} \in [1/2, 1]$ be such that $\sum_{i=1}^n h(x_i, \tilde{\alpha})^2 \leq (1 + \gamma) \sum_{i=1}^n h(x_i, \alpha)^2$ for all $\alpha \in [1/2, 1]$, where $\gamma < 1.38$. Then

$$\max_i |h(x_i, \tilde{\alpha})| \leq E \max_i x_i^2 \text{ for some constant } E < 2.95.$$

Proof. The proof of this claim follows from the same line of arguments as used in the proof of Claim 3, and then we apply the distance bound of Claim 4 to get the final result. By carefully analyzing the monotone behaviors of $|h(x, \alpha)|$ for $x \in [-1/4, 1/4]$ and $\alpha \in [1/2, 1]$ as in the proof of Claim 3, we get that

$$\frac{\max_i |h(x_i, \tilde{\alpha})|}{\max_i x_i^2} = \max_{M \in (0, 1/4]} \frac{|h(-M, \tilde{\alpha})|}{M^2}$$

where $M = \max_i x_i^2$. Again, as in the proof of Claim 3, where we have showed that $a^* \leq \beta(M)$, because the function $|h(-M, \alpha)|$ is monotonically increasing with respect to α for $\alpha \in [1/2, 1]$, we need to determine how large $\tilde{\alpha}$ can be. Using the result of Claim 4 and the assumption that $\gamma < 1.38$, we get

$$\tilde{\alpha} \leq \alpha^* + 0.51\sqrt{1.38}M \leq \beta(M) + 0.6M.$$

Therefore, we maximize the worst-case ratio to get

$$\begin{aligned} \max_{M \in (0, 1/4]} \frac{|h(-M, \tilde{\alpha})|}{M^2} &\leq \max_{M \in (0, 1/4]} \frac{h(-M, \beta(M) + 0.6M)}{M^2} \\ &= \frac{h(-1/4, \beta(1/4) + 3/20)}{1/16} \\ &= \frac{157}{\sqrt{3}} - \frac{84187}{960} < 2.95. \end{aligned}$$

This finishes the proof. $\square$

C.2. Proof of Theorem 4.1

Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be such that $\|\mathbf{A}\|_2 \leq 1$ and $\mathbf{A}^2$ is symmetric. Let $\mathbf{X}_0 = \mathbf{A}$ and let $\mathbf{X}_1, \mathbf{X}_2, \dots$ be the sequence generated by Equation (2) with $d = 1$, where α_k is determined by Equation (3) with $\ell = 1/2$ and $u = 1$. Denote $\mathbf{R}_k = \mathbf{I} - \mathbf{X}_k^2$. Since $d = 1$, Equation (2) simplifies to

$$\mathbf{X}_{k+1} = \mathbf{X}_k(\mathbf{I} + \alpha_k(\mathbf{I} - \mathbf{X}_k^2)) = \mathbf{X}_k(\mathbf{I} + \alpha_k \mathbf{R}_k),$$

and hence

$$\mathbf{R}_{k+1} = \mathbf{I} - \mathbf{X}_{k+1}^2 = \mathbf{I} - (\mathbf{I} - \mathbf{R}_k)(\mathbf{I} + \alpha_k \mathbf{R}_k)^2.$$

Define $h(x, \alpha) = 1 - (1 - x)(1 + \alpha x)^2$ so that we can write the above recurrence relation with respect to $\mathbf{R}_k$ succinctly as

$$\mathbf{R}_{k+1} = h(\mathbf{R}_k, \alpha_k).$$

In order to see that

$$\|\mathbf{R}_k\|_2 \leq \|\mathbf{R}_0\|_2^{2^{k-2}},$$

where α_k is computed according to Equation (3), we rely on the properties of h in Lemma C.1. Because $\mathbf{A}^2$ is symmetric, $\mathbf{R}_0 = \mathbf{I} - \mathbf{A}^2$ is also symmetric. Since $\mathbf{R}_{k+1} = h(\mathbf{R}_k, \alpha_k)$ and $h(x, \alpha)$ is a polynomial in x , it follows that $\mathbf{R}_k$ is symmetric for all k . Therefore,

$$\|\mathbf{R}_k\|_2 = \max_i |\lambda_{k,i}| \text{ for all } k,$$

where $\lambda_{k,i}$ denotes the i -th eigenvalue of $\mathbf{R}_k$. We will assume without loss of generality that the eigenvalues are ordered in a way such that $\lambda_{k+1,i} = h(\lambda_{k,i}, \alpha_k)$. For $k = 0$, because $\|\mathbf{X}_0\|_2 \leq 1$ and $\mathbf{X}_0^2$ is symmetric, the eigenvalues of $\mathbf{X}_0^2$ are all real-valued and lie in the interval $[0, 1]$, and therefore $0 \leq \lambda_{0,i} < 1$ for all i . Using Lemma C.1, we get that

- $\|\mathbf{R}_{k+1}\|_2 \leq \|\mathbf{R}_k\|_2^2$ if $\|\mathbf{R}_k\|_2 \geq 1/2$;

- $\|\mathbf{R}_{k+1}\|_2 \leq 1/4$ if $\|\mathbf{R}_k\|_2 \leq 1/2$;
- $\|\mathbf{R}_{k+1}\|_2 \leq 1.71\|\mathbf{R}_k\|_2^2$ if $\|\mathbf{R}_k\|_2 \leq 1/4$.

Let k_1 be such that $\|\mathbf{R}_{k_1}\|_2 \leq 1/4 < \|\mathbf{R}_{k_1-1}\|_2$. Because $\|\mathbf{R}_{k_1-1}\|_2 > 1/4$, we must have

$$\|\mathbf{R}_{k_1-2}\|_2 \geq \sqrt{\|\mathbf{R}_{k_1-1}\|_2} > \sqrt{1/4} = 1/2 > 1.71\|\mathbf{R}_{k_1}\|_2.$$

Then by induction we have that for $k_2 \geq 0$,

$$\|\mathbf{R}_{k_1+k_2}\|_2 \leq \left(1.71\|\mathbf{R}_{k_1}\|_2\right)^{2^{k_2}} \leq \left(\|\mathbf{R}_{k_1-2}\|_2\right)^{2^{k_2}} \leq \|\mathbf{R}_0\|_2^{2^{k_1+k_2-2}}.$$

Finally, the convergence to $\text{sign}(\mathbf{A})$ can be established following the same argument as in the proof of Theorem 3.1 and Theorem 5.2 of [Kenney & Laub \(1991\)](#). For completeness we repeat the main arguments below. Let

$$S = \{x : |1 - x^2| < 1\}, S_+ = \{x \in S : \text{Re}(x) > 0\}, S_- = \{x \in S : \text{Re}(x) < 0\}.$$

Denote

$$p_{k,d}(x) = xg_d(1 - x^2; \alpha_k).$$

Let $x_{k,i}$ denote the i -th eigenvalue of $\mathbf{X}_k$ and assume without loss of generality that the indices are ordered such that $x_{k+1,i} = p_{k,d}(x_{k,i})$. Then since $\|\mathbf{R}_k\|_2 < 1$ for all k , we get that

$$1 - x_{k,i}^2 < 1 \text{ and } 1 - p_{k,d}(x_{k,i})^2 = 1 - x_{k+1,i}^2 < 1, \text{ for all } k.$$

So $p_{k,d}$ maps S into S for all k . Since $S_+ \cap S_- = \emptyset$ and each is a connected set, $p_{k,d}(S_+)$ must lie entirely in either S_+ or S_- , because $p_{k,d}$ is a continuous mapping. But since $1 \in S_+$ and $p_{k,d}(1) = 1$, we must have $p_{k,d}(S_+) \subseteq S_+$ for all k . Similarly, $p_{k,d}(S_-) \subseteq S_-$ for all k . Thus, by induction, we have that if $x_{0,i} \in S_+$ then $x_{k,i} \in S_+$ for all k . Since $\lim_{k \rightarrow +\infty} \|\mathbf{R}_k\|_2 = 0$, which means $\lim_{k \rightarrow +\infty} 1 - x_{k,i}^2 = 0$ for all i , we must have

$$\lim_{k \rightarrow +\infty} x_{k,i} = \text{sign}(x_{0,i})$$

for all i . Then, using Lemma 5.1 of [Kenney & Laub \(1991\)](#) and the definition of matrix sign in terms of the Jordan form, we get $\mathbf{X}_k \rightarrow \text{sign}(\mathbf{X}_0)$.

C.3. Proof of Theorem 4.2

The basic setup is the same as in the proof of Theorem 4.1. For the reader's convenience, we repeat the same setup here. Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be such that $\|\mathbf{A}\|_2 \leq 1$ and $\mathbf{A}^2$ is symmetric. Let $\mathbf{X}_0 = \mathbf{A}$ and let $\mathbf{X}_1, \mathbf{X}_2, \dots$ be the sequence generated by Equation (2) with $d = 1$, where α_k is determined by Equation (4) with $\ell = 1/2$ and $u = 1$. With a slight abuse of notation, we will use ℓ as indices of iteration counter from now on. Fix $k \geq 0$ and let $0 \leq \ell \leq k$. Denote $\mathbf{R}_\ell = \mathbf{I} - \mathbf{X}_\ell^2$. Since $d = 1$, Equation (2) simplifies to

$$\mathbf{X}_{\ell+1} = \mathbf{X}_\ell(\mathbf{I} + \alpha_\ell(\mathbf{I} - \mathbf{X}_\ell^2)) = \mathbf{X}_\ell(\mathbf{I} + \alpha_\ell \mathbf{R}_\ell),$$

and hence

$$\mathbf{R}_{\ell+1} = \mathbf{I} - \mathbf{X}_{\ell+1}^2 = \mathbf{I} - (\mathbf{I} - \mathbf{R}_\ell)(\mathbf{I} + \alpha_\ell \mathbf{R}_\ell)^2.$$

As in the proof of Theorem 4.1, define $h(x, \alpha) = 1 - (1 - x)(1 + \alpha x)^2$ so that we may write the recurrence relation with respect to $\mathbf{R}_\ell$ succinctly as

$$\mathbf{R}_{\ell+1} = h(\mathbf{R}_\ell, \alpha_\ell).$$

In order to see that

$$\|\mathbf{R}_k\|_2 \leq \|\mathbf{R}_0\|_2^{2^{k-3}},$$

when α_ℓ , $0 \leq \ell \leq k$, is computed as in Equation (4), we rely again on the properties of h in Lemma C.1. Because $\mathbf{A}^2$ is symmetric, $\mathbf{R}_0 = \mathbf{I} - \mathbf{A}^2$ is also symmetric. Since $\mathbf{R}_{k+1} = h(\mathbf{R}_k, \alpha_k)$ and $h(x, \alpha)$ is a polynomial in x , it follows that $\mathbf{R}_\ell$ is symmetric for all ℓ . Therefore,

$$\|\mathbf{R}_\ell\|_2 = \max_i |\lambda_{\ell,i}| \text{ for all } \ell,$$

where $\lambda_{\ell,i}$ denotes the i -th eigenvalue of $\mathbf{R}_\ell$. We will assume without loss of generality that the eigenvalues are ordered in such a way that $\lambda_{\ell+1,i} = h(\lambda_{\ell,i}, \alpha_\ell)$. For $\ell = 0$, because $\|\mathbf{X}_0\|_2 \leq 1$ and $\mathbf{X}_0^2$ is symmetric, the eigenvalues of $\mathbf{X}_0^2$ are all real-valued and lie in the interval $[0, 1]$, and therefore $0 \leq \lambda_{0,i} < 1$ for all i .

Let $\mathbf{S}_\ell \in \mathbb{R}^{p \times n}$ be random matrices consisting of i.i.d Gaussian entries $[\mathbf{S}_\ell]_{i,j} \sim \mathcal{N}(1, 1/p)$ with $p \geq 48(\log n + \log(1/\delta) + \log k + 41.4)$. Using standard result in randomized numerical linear algebra, for example, Proposition 3.7 in Balabanov & Nouy (2019), we know that $\mathbf{S}_\ell$ is a $(6, \epsilon, \frac{\delta}{kn})$ -OSE for $\epsilon = 0.405$. Now fix $\ell \in \{0, 1, \dots, k\}$ and $\mathbf{R}_\ell$. Let $\mathbf{r}_\ell^{(i)}$ denote the i -th column of $\mathbf{R}_\ell$, and let $\mathbf{r}_{\ell+1}^{(i)}$ denote the i -th column of $\mathbf{R}_{\ell+1} = h(\mathbf{R}_\ell, \alpha)$ for any α . Since $\mathbf{R}_{\ell+1} = h(\mathbf{R}_\ell, \alpha)$ is a degree-5 polynomial with respect to $\mathbf{R}_\ell$, we get $\mathbf{r}_{\ell+1}^{(i)} \in \text{span}\{\mathbf{e}_i, \mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_5\}$ where $\mathbf{e}_i$ is the i -th standard basis vector, and $\mathbf{v}_j$ is the j -th column of $\mathbf{R}_\ell^j$. This means that each $\mathbf{r}_{\ell+1}^{(i)}$ lives in a 6-dimensional subspace of $\mathbb{R}^n$. Since we can express the squared Frobenius norm as the sum of squared column ℓ_2 norms

$$\|\mathbf{R}_{\ell+1}\|_F^2 = \sum_{i=1}^n \|\mathbf{r}_{\ell+1}^{(i)}\|_2^2,$$

we use the $(6, \epsilon, \frac{\delta}{kn})$ -OSE property of $\mathbf{S}_\ell$ and a union bound over $i \in \{1, 2, \dots, n\}$ to get, with probability at least $1 - \delta/k$,

$$(1 - \epsilon)\|\mathbf{R}_{\ell+1}\|_F^2 = \sum_{i=1}^n (1 - \epsilon)\|\mathbf{r}_{\ell+1}^{(i)}\|_2^2 \leq \sum_{i=1}^n \|\mathbf{S}_\ell \mathbf{r}_{\ell+1}^{(i)}\|_2^2 = \|\mathbf{S}_\ell \mathbf{R}_{\ell+1}\|_F^2$$

and

$$(1 + \epsilon)\|\mathbf{R}_{\ell+1}\|_F^2 = \sum_{i=1}^n (1 + \epsilon)\|\mathbf{r}_{\ell+1}^{(i)}\|_2^2 \geq \sum_{i=1}^n \|\mathbf{S}_\ell \mathbf{r}_{\ell+1}^{(i)}\|_2^2 = \|\mathbf{S}_\ell \mathbf{R}_{\ell+1}\|_F^2.$$

Let α_ℓ^* and $\tilde{\alpha}_\ell$ be computed according to Equation (3) and Equation (4), respectively, that is

$$\alpha_\ell^* = \arg \min_{\alpha \in [1/2, 1]} \|h(\mathbf{R}_\ell, \alpha)\|_F^2, \quad \tilde{\alpha}_\ell = \arg \min_{\alpha \in [1/2, 1]} \|\mathbf{S}_\ell h(\mathbf{R}_\ell, \alpha)\|_F^2,$$

then we get that, with probability at least $1 - \delta/k$,

$$(1 - \epsilon)\|h(\mathbf{R}_\ell, \tilde{\alpha}_\ell)\|_F^2 \leq \|\mathbf{S}_\ell h(\mathbf{R}_\ell, \tilde{\alpha}_\ell)\|_F^2 \leq \|\mathbf{S}_\ell h(\mathbf{R}_\ell, \alpha_\ell^*)\|_F^2 \leq (1 + \epsilon)\|h(\mathbf{R}_\ell, \alpha_\ell^*)\|_F^2,$$

and hence

$$\|h(\mathbf{R}_\ell, \tilde{\alpha}_\ell)\|_F^2 \leq \left(\frac{1 + \epsilon}{1 - \epsilon}\right) \|h(\mathbf{R}_\ell, \alpha_\ell^*)\|_F^2 < (1 + \gamma) \|h(\mathbf{R}_\ell, \alpha_\ell^*)\|_F^2$$

where $\gamma < 1.37$ for $\epsilon = 0.405$. Since

$$\|h(\mathbf{R}_\ell, \tilde{\alpha})\|_F^2 = \sum_{i=1}^n h(\lambda_{\ell,i}, \tilde{\alpha})^2,$$

we may apply Lemma C.1 and get that, for each $0 \leq \ell \leq k$,

- $\|\mathbf{R}_{\ell+1}\|_2 \leq \|\mathbf{R}_\ell\|_2^2$ if $\|\mathbf{R}_\ell\|_2 \geq 1/2$;
- $\|\mathbf{R}_{\ell+1}\|_2 \leq 1/4$ if $\|\mathbf{R}_\ell\|_2 \leq 1/2$;
- $\|\mathbf{R}_{\ell+1}\|_2 \leq 2.95\|\mathbf{R}_\ell\|_2^2$ with probability at least $1 - \delta/k$.

Let ℓ_1 be such that $\|\mathbf{R}_{\ell_1}\|_2 \leq 1/4 < \|\mathbf{R}_{\ell_1-1}\|_2$. Because

$$h(h(0.75, 0.5), 0.5) < 0.25 < \|\mathbf{R}_{\ell_1-1}\|_2,$$

using the monotone properties of $h(x, \alpha)$ for $x \in [1/2, 1]$ and $\alpha \in [1/2, 1]$ from the proof of Claim 1, we must have

$$\|\mathbf{R}_{\ell_1-3}\|_2 > 0.75 > 2.95\|\mathbf{R}_{\ell_1}\|_2.$$

Then by induction and a union bound for $0 \leq \ell_2 \leq k$, we get that with probability at least $1 - \delta$,

$$\|\mathbf{R}_{\ell_1+\ell_2}\|_2 \leq \left(2.95\|\mathbf{R}_{\ell_1}\|_2\right)^{2^{\ell_2}} \leq \left(\|\mathbf{R}_{\ell_1-3}\|_2\right)^{2^{\ell_2}} \leq \|\mathbf{R}_0\|_2^{2^{\ell_1+\ell_2-3}},$$

for $1 \leq \ell_1 + \ell_2 \leq k$. This proves the required result for the rate of convergence. The convergence to $\text{sign}(\mathbf{A})$ follows exactly the same way as before.

D. On Coefficient Bound $[\ell, u]$ for Polynomial Optimization in Higher-Order Newton-Schulz Variants

When using PRISM to accelerate the convergence of Newton-Schulz iteration for matrix sign, polar decomposition, and square roots, one needs to enforce a bound $[\ell, u]$ on the polynomial coefficient α_k . We now discuss how to choose this bound in a principled way.

Let us start by reminding the reader with the notations that we used in Appendix A.1 for the relevant Newton-Schulz iterations. We write $f(\xi) = (1 - \xi)^{-1/2}$ and $f_d(\xi)$ the d -th order Taylor approximation of $f(\xi)$ around $\xi = 0$. Most notably, we have

$$f_1(\xi) = 1 + 0.5\xi, \quad f_2(\xi) = 1 + 0.5\xi + 0.375\xi^2,$$

which give rise to the 3rd- and 5th-order Newton-Schulz iteration, respectively, for the matrix sign, and hence for polar decomposition and square roots due to their connections with the matrix sign (cf. Theorem 5.2 and Theorem 5.1). In PRISM, we replace $f_d(\xi)$ with $g_d(\xi; \alpha) = f_{d-1}(\xi) + \alpha\xi^d$. For example,

$$g_1(\xi; \alpha) = 1 + \alpha\xi, \quad g_2(\xi; \alpha) = 1 + 0.5\xi + \alpha\xi^2.$$

For computing the matrix sign function with Newton-Schulz, at the k -th iteration, given the current iterate $\mathbf{X}_k$, residual matrix $\mathbf{R}_k = \mathbf{I} - \mathbf{X}_k^2$, and sketch matrix $\mathbf{S}$, finding a good value of α_k by naively minimizing

$$m(\alpha) = \left\| \mathbf{S} \left(\mathbf{I} - \mathbf{X}_k^2 g_d(\mathbf{R}_k; \alpha) \right) \right\|_F^2$$

may lead to performance degradation. To address this, for the case $d = 1$, in Section 4 we suggested minimizing $m(\alpha)$ over an interval $\alpha \in [\ell, u]$, where $\ell = 0.5$ and $u = 1$. In Appendix C, we showed that, by constraining α_k in this way, the resulting polynomial has some good properties that enable us to prove Theorem 4.1 and Theorem 4.2, respectively, which state that PRISM-based 3rd-order Newton-Schulz converges quadratically in the worst case, matching the convergence rate of classical Newton-Schulz. The proof relies on Lemma C.1, which establishes several desired properties for the polynomial

$$h(x, \alpha) = 1 - (1 - x)(1 + \alpha x)^2.$$

This polynomial arises from the PRISM-based 3-rd order matrix sign iteration $\mathbf{X}_{k+1} = \mathbf{X}_k(\mathbf{I} + \alpha_k \mathbf{R}_k)$ where $\mathbf{R}_k = \mathbf{I} - \mathbf{X}_k^2$. One may easily verify that $\mathbf{R}_{k+1}$ and $\mathbf{R}_k$ are related by the recurrence relation

$$\mathbf{R}_{k+1} = h(\mathbf{R}_k, \alpha_k).$$

Because the residual matrix is symmetric (recall our assumption that the input matrix $\mathbf{A}$ satisfies $\mathbf{A}^2$ is symmetric), roughly speaking, the polynomial $h(x, \alpha)$ models how each individual residual value x (think of an eigenvalue of the residual matrix) progresses from one iteration to the next, given the coefficient α in the current recurrence relation. For a precise treatment, we refer the reader to Appendix C.2. Hence, in rough words, the properties that we would like $h(x, \alpha)$ to have are the following:

1. Initial phase:

- *Large residuals contract rapidly:* If x is “large”, then $h(x, \alpha)$ should be contracting towards 0 at the desired rate, for any α in the interval $[\ell, u]$. See, for example, Item 1 in Lemma C.1.
- *Small residuals remain small:* If x is “small”, then $h(x, \alpha)$ should remain small, for any α in the interval $[\ell, u]$. See, for example, Item 2 in Lemma C.1.

2. Convergence phase:

- *Point-wise convergence with optimal α :* Let $\{x_1, x_2, \dots, x_n\}$ be the set of all residuals (think of the set of all eigenvalues of the residual matrix). Then $h(x_i, \alpha^*)$ should be converging towards 0 at the desired rate, for all i , where $\alpha^* = \arg \min_{\alpha \in [\ell, u]} \sum_{i=1}^n h(x_i, \alpha)^2$. See, for example, Item 3 of Lemma C.1.

In general, for any $d \geq 1$, the bound $[\ell, u]$ that constrains the coefficient α in $g_d(\xi; \alpha)$ should be set to allow these properties. We have shown an explicit example for the case $d = 1$ in Lemma C.1. It enables us to prove Theorem 4.1 on the worst-case convergence guarantee for PRISM-based 3rd-order Newton-Schulz for matrix sign (which readily applies to polar

decomposition and square roots computation). Therefore, in Algorithm 1 and Algorithm 3, we set the input parameters to be $\ell = 0.5$ and $u = 1$. Note that these are universal constants in the sense that they do not depend on the spectral properties (or any other characteristics such as the size) of the input matrix.

For $d \geq 2$ that gives rise to higher-order PRISM Newton-Schulz variants, we can repeat the same line of reasoning and find appropriate choices for $[\ell, u]$. In the next, we give an example on how to do this for $d = 2$. The resulting $[\ell, u]$ is used in Algorithm 2 and Algorithm 4 for the PRISM-based 5th-order Newton-Schulz for polar decomposition and square root computation, respectively.

D.1. Choosing $[\ell, u]$ for PRISM-based 5th-order Newton-Schulz and beyond

The PRISM-based 5th-order Newton-Schulz iteration for matrix sign (and similarly for polar decomposition and square root computation, due to Theorem 5.2 and Theorem 5.1) is given as

$$\mathbf{R}_k = \mathbf{I} - \mathbf{X}_k^2, \quad \mathbf{X}_{k+1} = \mathbf{X}_k(\mathbf{I} + 0.5\mathbf{R}_k + \alpha_k\mathbf{R}_k^2).$$

Hence $\mathbf{R}_{k+1}$ and $\mathbf{R}_k$ are related by the recurrence relation

$$\mathbf{R}_{k+1} = \mathbf{I} - \mathbf{X}_{k+1}^2 = \mathbf{I} - (\mathbf{I} - \mathbf{R}_k)(\mathbf{I} + 0.5\mathbf{R}_k + \alpha_k\mathbf{R}_k^2)^2.$$

The polynomial of interest in this case is thus

$$h(x, \alpha) = 1 - (1 - x) \left(1 + \frac{1}{2}x + \alpha x^2\right)^2.$$

By applying a similar line of reasoning on the analysis of monotone behaviors of $h(x, \alpha)$ and its derivatives, we can show that $h(x, \alpha)$ have the following properties:

1. *Large residuals contract rapidly:* $h(x, \alpha) \in [-3/8, x^3]$ for all $x \in [3/4, 1]$ and for all $\alpha \in [3/8, 29/20]$;
2. *Small residuals remain small:* $h(x, \alpha) \in [-3/8, 3/8]$ for all $x \in [-3/8, 3/4]$ and for all $\alpha \in [3/8, 29/20]$;
3. Let $n \geq 1$ and $x_1, x_2, \dots, x_n \in [-3/8, 3/8]$ and $\alpha^* = \arg \min_{\alpha \in [3/8, 29/20]} \sum_{i=1}^n h(x_i, \alpha)^2$, we have

$$\max_i |h(x_i, \alpha^*)| \leq C \max_i |x_i|^3 \text{ for some constant } C < 1.7.$$

By following a similar argument in Appendix C.2 and using the above properties on $h(x, \alpha)$, one can show that, if we set $\ell = 0.375$ and $u = 1.45$, then PRISM-based 5th-order Newton-Schulz is cubically convergent in the worst case, matching the rate of classical 5th-order Newton-Schulz. This gives the worst-case convergence guarantee for Algorithm 2 and Algorithm 4, and explains why we choose ℓ and u to be these values.

This should in principle apply to any d . For example, for $d = 3$, which corresponds to the 7th-order Newton-Schulz, one may analogously show that $\ell = 5/16$ and $u = 1$ is a good choice. In practice, these bounds need not to be tight. For example, for the PRISM-based 5th-order Newton-Schulz iteration, we find that setting $\ell = 3/8$ and $u = 1$ yields similar performance. The general rule is that the interval $[\ell, u]$ should include the original coefficient of the Taylor polynomial. In approximating $f(\xi) = (1 - \xi)^{-1/2}$, since the Taylor polynomials $f_d(\xi)$ approximate $f(\xi)$ from the below, we simply set the lower bound ℓ to be value of the coefficient in the corresponding Taylor polynomial. For example, for $d = 1$, we set $\ell = 0.5$ as $f_1(\xi) = 1 + 0.5\xi$; for $d = 2$, we set $\ell = 0.375$ as $f_2(\xi) = 1 + 0.5\xi + 0.375\xi^2$.

E. Details of Numerical Experiments

The empirical evaluation of polar decomposition algorithms for Gaussian random matrices and HTMP random matrices is run in single precision (i.e., `torch.float32`) on an Nvidia A100 GPU. For the numerical experiment in Figure 4, the matrix $\mathbf{A} \in \mathbb{R}^{n \times m}$ has size $n = 8000$ and $m = 4000$.

In the experiment with the Shampoo optimizer, we made the standard ResNet-20 and ResNet-32 slightly larger to demonstrate more clearly the difference between different algorithms for matrix square root when the input matrix has reasonably large size, e.g., larger than 100. For both ResNet-20 and ResNet-32, we kept the stride at one for each convolutional layer; for

ResNet-20, we additionally removed the average pooling layer before the final fully connected layer. We set the maximum preconditioner dimension to 2048 in the Distributed Shampoo optimizer (Shi et al., 2023), so the matrices of which we need to compute the inverse square root have dimension at most 2048×2048 . We also tested setting the maximum preconditioner dimension to 1024 and 4096, respectively, and the results are similar. Generally, we find that the larger dimension the preconditioner has, the better performance PRISM has relative to eigenvalue decomposition and PolarExpress. We set the learning rate to 0.001 and the weight decay to 0.0005. We did not tune hyperparameters for this experiment. In this experiment we used the 5th-order Newton-Schulz iteration accelerated by PRISM. Other variants of Newton-Schulz, such as the one that uses a degree-3 polynomial update at each iteration, can also be accelerated by PRISM. In our experiment, we also tried to use five iterations of this variant to compute the inverse square root of Shampoo’s preconditioners, and we got similar results. The PolarExpress algorithm that we use in our experiment is the one that is optimized for $\sigma_{\min} = 10^{-3}$ (i.e. Algorithm 1 of Amsel et al. (2026)).

In the experiment with Muon optimizer in Figure 6, we use PolarExpress (Algorithm 1 of Amsel et al. (2026)), PRISM-based Newton-Schulz iteration with degree-5 polynomial (PRISM-5), and PRISM-based Newton-Schulz with degree-3 polynomial (PRISM-3) to compute the polar decomposition of the gradient matrices. In this experiment, we use five iterations for PolarExpress and PRISM-3, and three iterations for PRISM-5. Empirically, we observed that PRISM-5 with 3 iterations attains lower average residual error than PolarExpress with 5 iterations. For instance, at final step in Figure 6, the average Frobenius norm error over all layers for PRISM-5 with 3 iterations and PolarExpress with 5 iterations are 12.7 and 23.4. Hence, for the computational efficiency, we only take 3 iterations for PRISM-5 at each step of training. In addition, based on the computed α_k in Figure 4, we observe that at the initial several iterations, the coefficient α_k is attained at the upper bound u in (4). Hence, when we implement PRISM-3 and PRISM-5 in Figure 6, we decide to use the highest value of α_k for the initial three iterations for efficiency. This means that we set $\alpha_k = 1$ for PRISM-3 and $\alpha_k = 29/20$ for PRISM-5 for the first three iterations. Note that, by Lemma C.1, setting α_k in this way preserves the initial quadratic convergence of PRISM-3. Additionally, we choose weight decay 0.01, momentum parameter 0.95, and initial learning rate 6×10^{-3} , and micro-batch size 4. We also compare these experiments with the baseline using AdamW with initial learning rate 3×10^{-4} and weight decay 0.1. All experiments are run on NVIDIA A100-SXM4-80GB with global batch size 32.

F. Additional Empirical Results

- Figure 7 and Figure 8 show the convergence of degree-5 polynomial methods with respect to the number of iterations. These figures compensate for Figure 3 and Figure 4, respectively, where we illustrate the convergence of algorithms with respect to the wall-clock time when running on a single Nvidia A100 GPU.
- Figure 9 and Figure 10 show the convergence of degree-5 polynomial methods for computing square roots.
- Figure 11 shows the accelerated convergence behavior of the PRISM-based DB Newton iteration (see Appendix A.2 for details). We observe that, when accelerated by PRISM, the Newton iteration can converge faster than the PRISM-based Newton-Schulz. However, because of the requirement to perform matrix inversion at every iteration, Newton iteration is generally less stable than the Newton-Schulz variants. An interesting future work is to exploit the faster convergence of PRISM-based Newton iteration while maintaining good numerical stability.

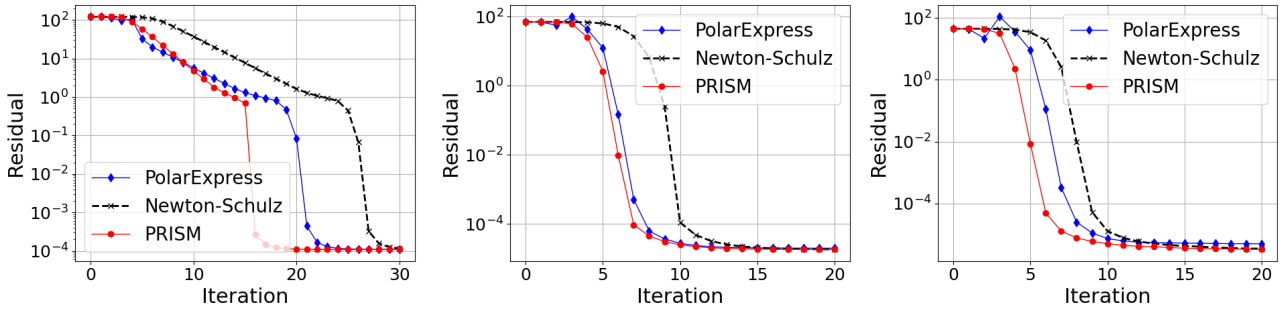


Figure 7. Convergence (with respect to iterations) of degree-5 polynomial methods for orthogonalizing a Gaussian random matrix $A \in \mathbb{R}^{n \times m}$ with varying aspect ratio $\gamma = n/m$. The figures show the Frobenius norm error $\|I - X_k^T X_k\|_F$ for $\gamma = 1, 4, 50$, from left to right respectively.

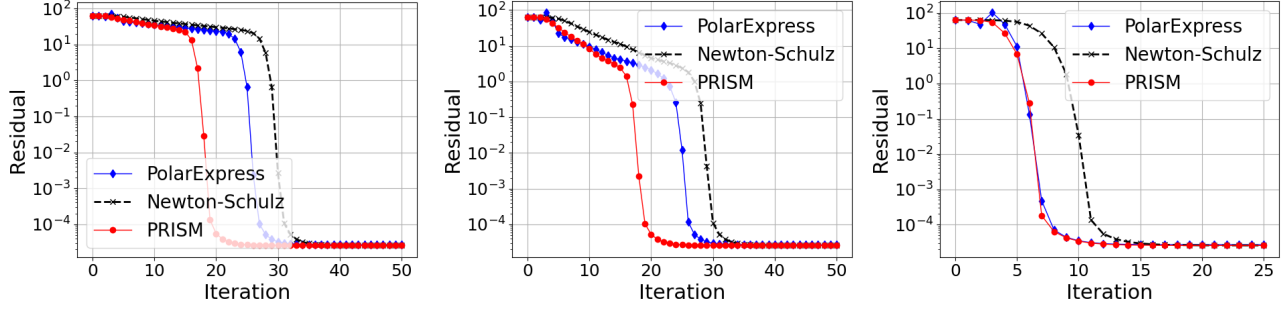


Figure 8. Convergence (with respect to iterations) of degree-5 polynomial methods for orthogonalizing random matrices generated by HTMP (Hodgkinson et al., 2025) with different parameter κ . Smaller κ indicates heavier tail in the spectral distribution. The figures show the Frobenius norm error $\|\mathbf{I} - \mathbf{X}_k^T \mathbf{X}_k\|_F$ for $\kappa = 0.1, 0.5, 100$, from left to right respectively.

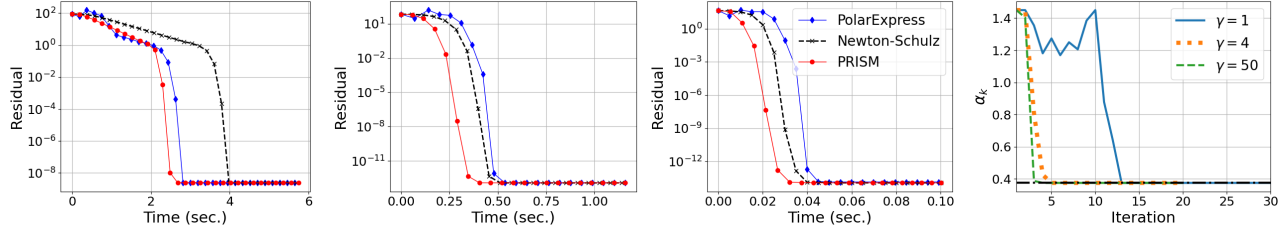


Figure 9. Convergence of degree-5 polynomial methods for computing the square root and inverse square root of $\mathbf{A} = \mathbf{G}^T \mathbf{G}$, where $\mathbf{G} \in \mathbb{R}^{n \times m}$ is a Gaussian random matrix with varying aspect ratio $\gamma = n/m$. That is, $\mathbf{A}$ is a Wishart matrix. The figures from left to right show the Frobenius norm error $\|\mathbf{I} - \mathbf{X}_k^{-2} \mathbf{A}\|_F$ for $\gamma = 1, 4, 50$, respectively. The last figure on the right shows the α_k 's computed by (4) in PRISM for different aspect ratios.

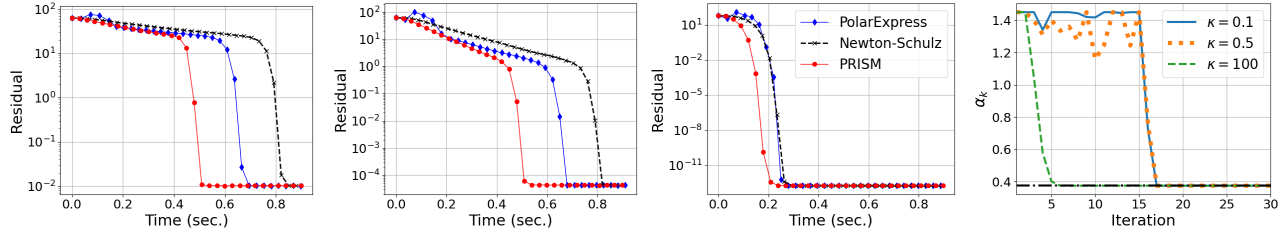


Figure 10. Convergence of degree-5 polynomial methods for computing the square root and inverse square root of $\mathbf{A} = \mathbf{G}^T \mathbf{G}$, where $\mathbf{G} \in \mathbb{R}^{n \times m}$ is a random matrix generated by HTMP (Hodgkinson et al., 2025) with different parameter κ . Smaller κ indicates heavier tail in the spectral distribution. The figures from left to right show the Frobenius norm error $\|\mathbf{I} - \mathbf{X}_k^{-2} \mathbf{A}\|_F$ for $\kappa = 0.1, 0.5, 100$, respectively. The last figure on the right shows the α_k 's computed by (4) in PRISM for different κ 's.

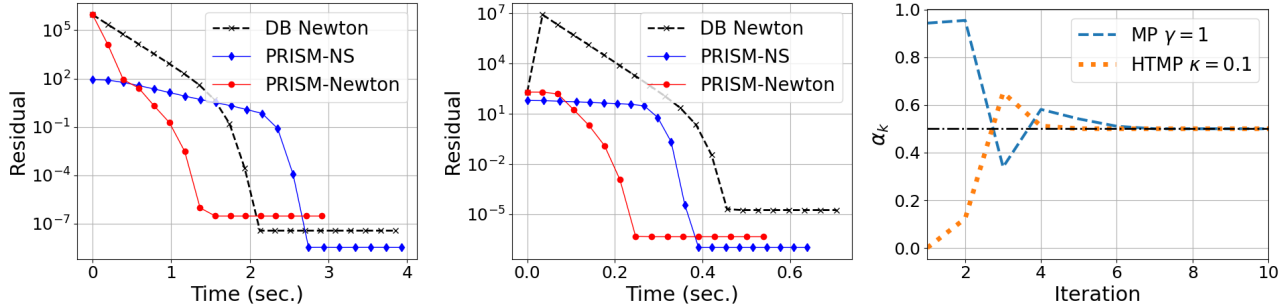


Figure 11. Convergence of PRISM-based DB Newton (PRISM-Newton, cf. Table 1 and Appendix A.2) for computing the square root and inverse square root. We compare with the classical DB Newton iteration. For reference we also compare with the PRISM-based Newton-Schulz (PRISM-NS) that we tested in the previous experiment. We select two representative input matrices from the previous experiment: (Left) Wishart matrix with aspect ratio $\gamma = 1$; (Middle) random matrix generated by HTMP with $\kappa = 0.1$. The rightmost plot shows the coefficient α_k computed by PRISM-Newton.